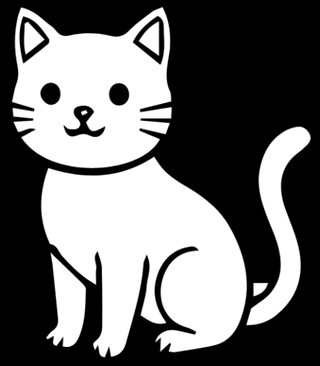
BRIEFINGS

black hat

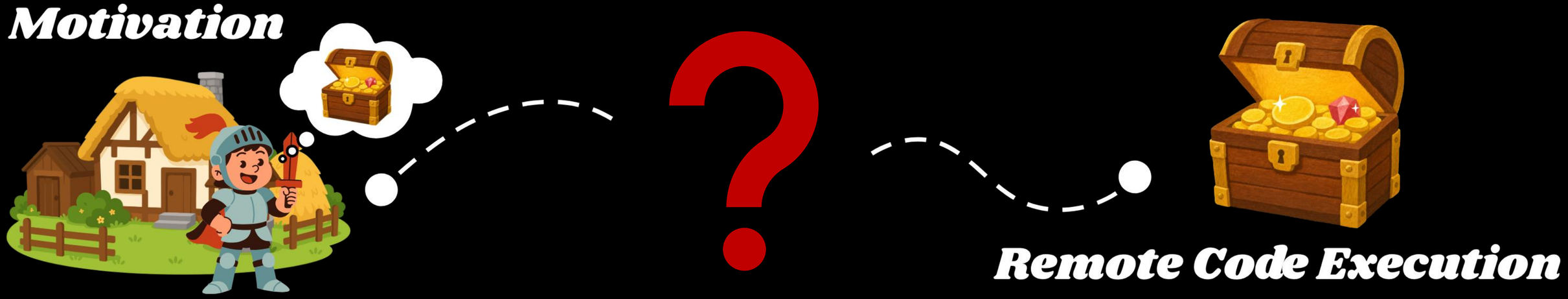


Batch me if you can

Breaking with the State-of-the-Art of Fuzzing Cryptographic Architectures

Niklas Vogel | Haya Schulmann
ATHENE @ Goethe University Frankfurt

Agenda: A journey towards RCE in RPKI



Agenda: A journey towards RCE in RPKI

Understanding the (fuzzing) challenges

Agenda: A journey towards RCE in RPKI

Building a fuzzer prototype

Agenda: A journey towards RCE in RPKI

Making it powerful

Agenda: A journey towards RCE in RPKI

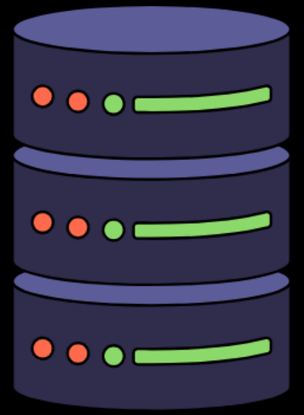
Remote Code Execution (and 7 other CVEs)

1 - Motivation

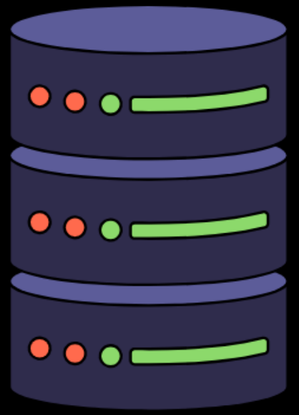
Fuzzing Cryptographic Architectures

Motivation: Cryptographic Architectures

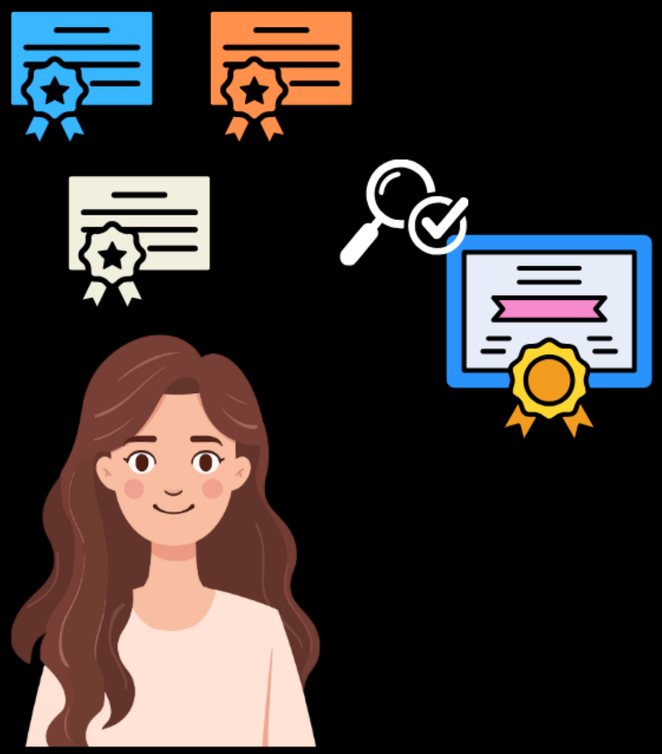
Motivation: Cryptographic Architectures

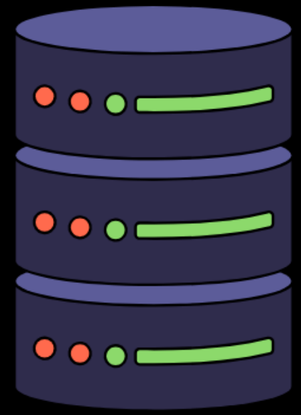


Motivation: Cryptographic Architectures

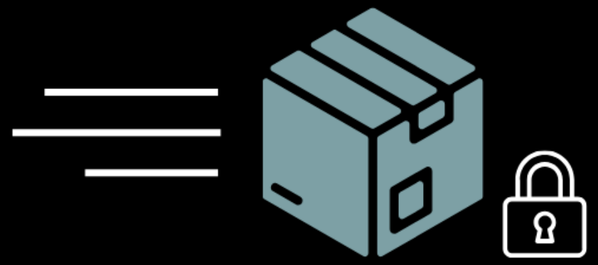


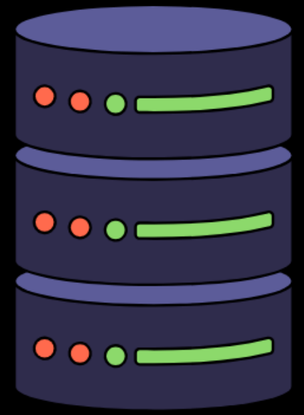
Motivation: Cryptographic Architectures



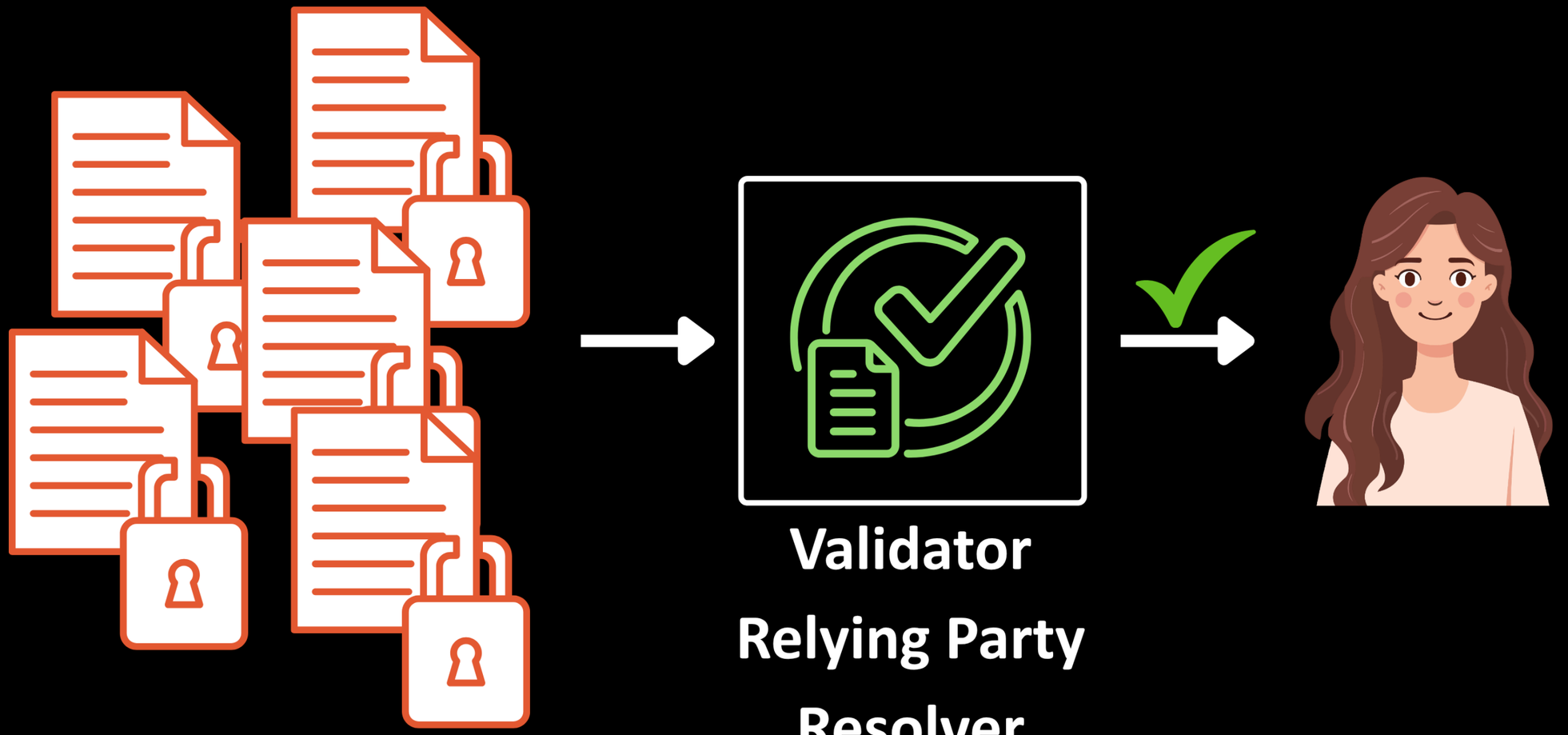


Motivation: Cryptographic Architectures



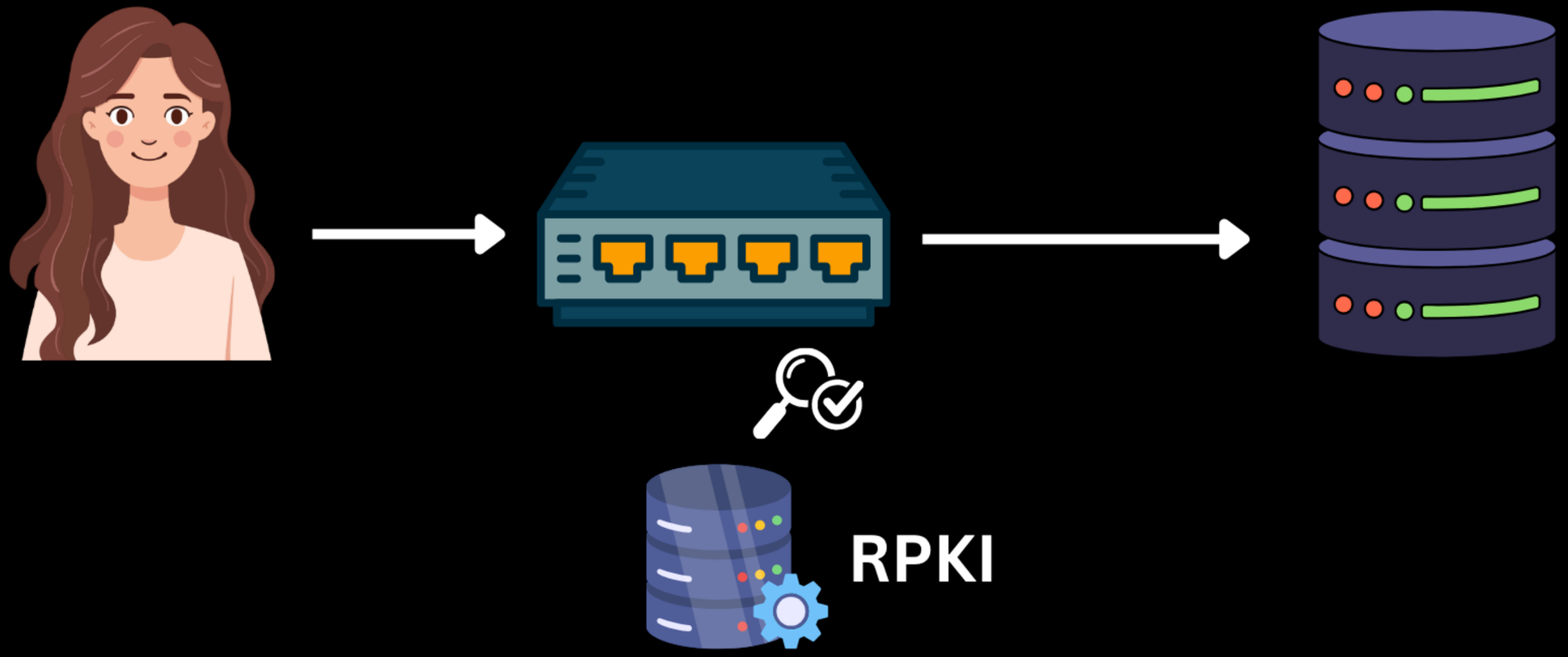


Cryptographic Validators



Motivation: Cryptographic Architectures

Motivation: Cryptographic Architectures



RPKI =

Routing Public Key Infrastructure*

RPKI =

[redacted] Public Key Infrastructure*

Resource

*for routing

RPKI =

[redacted] Public Key Infra[redacted]
Resource

*for routing

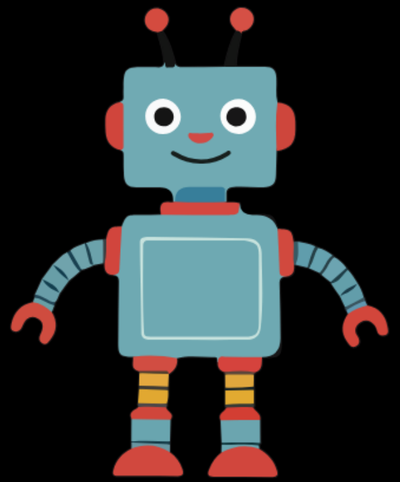
Testing Validators with Fuzzing

2 - Fuzzing Challenge

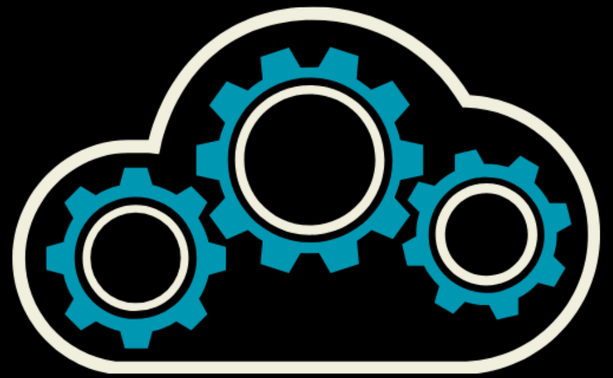
How to deal with cryptography?

Fuzzing

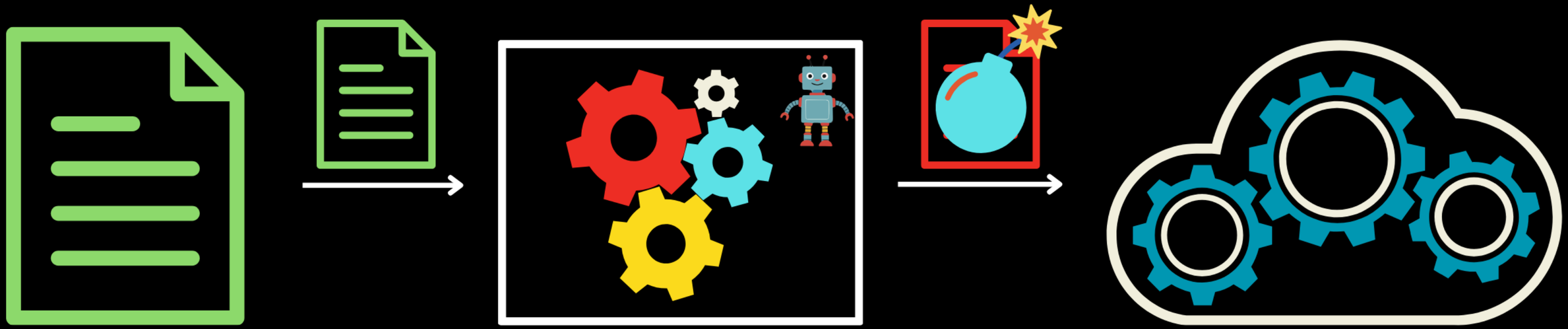
Fuzzer



Validator

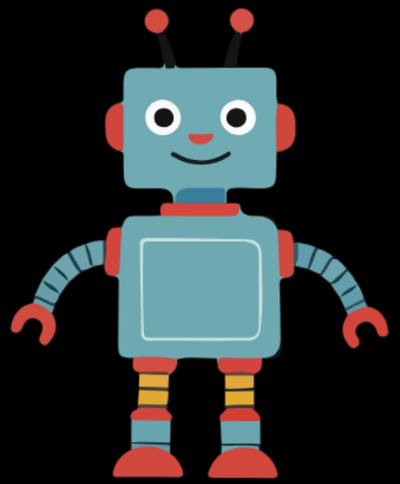


Fuzzing Architecture



Fuzzing cryptographic Architectures

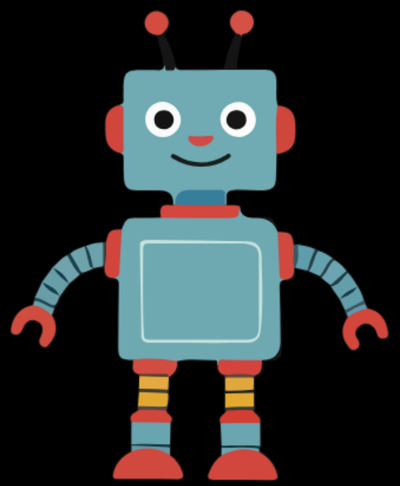
Fuzzer

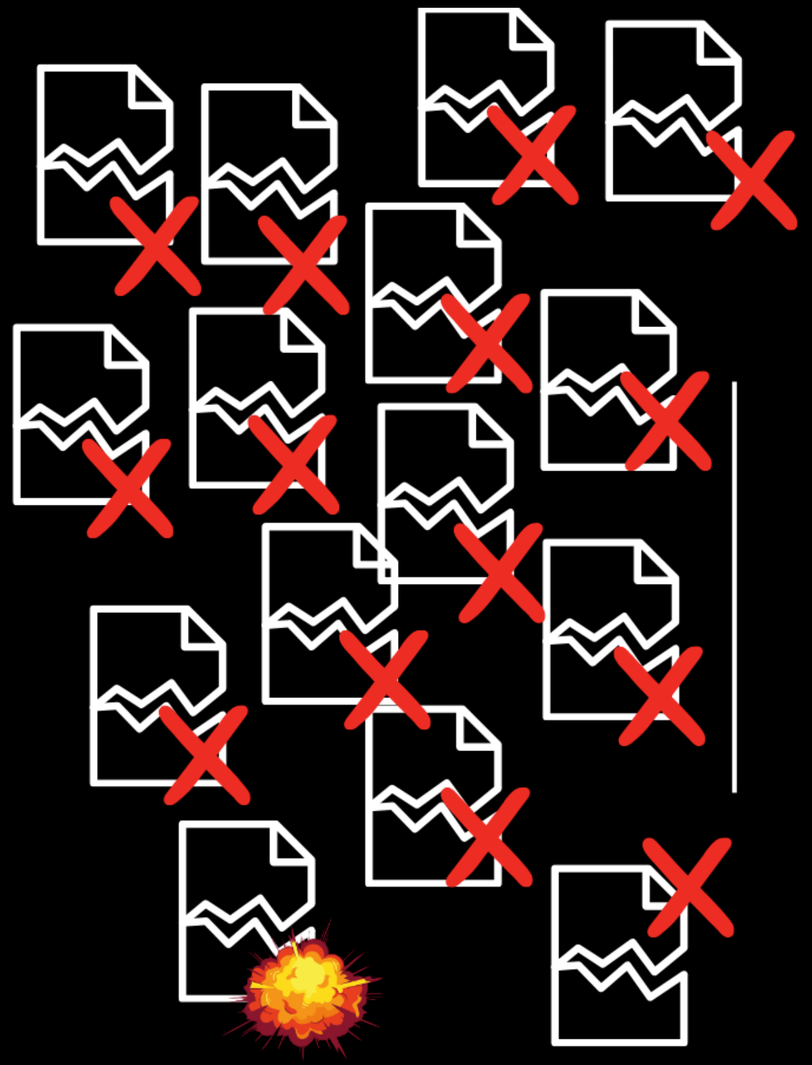


Validator

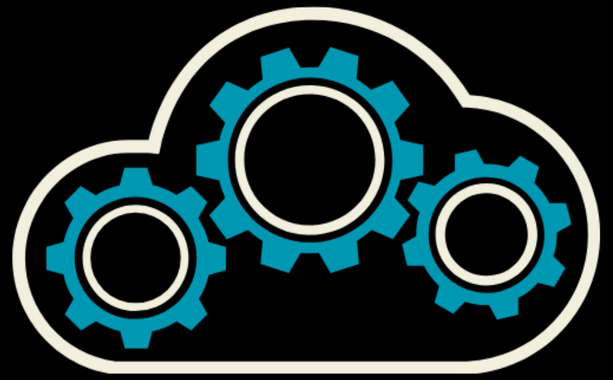
How to fuzz cryptographic Architectures?

Fuzzer





Validator



Guessing cryptographic Values

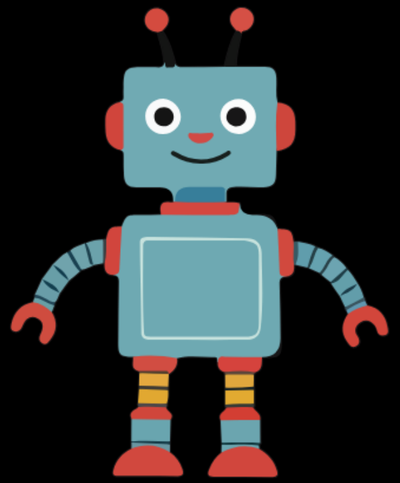
0.000

Guessing cryptographic Values

0.0000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000
0000000000000000000000000000000000
0000000000000000000000000000000000
0000000000000000000000000000000000
0000000000000000000000000000000000
0000000000000000000000000000000000
0000000000000000000000000000000000
0000000000000000000000000000000000000000[.......]1%

Removing Cryptography

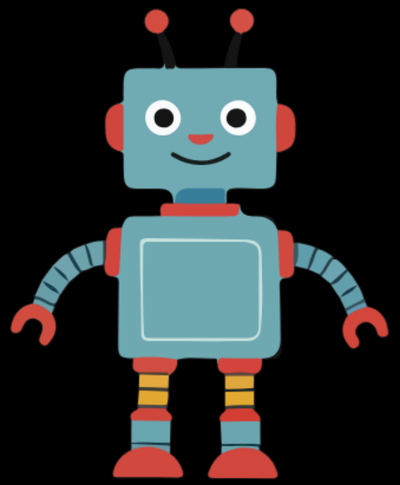
Fuzzer



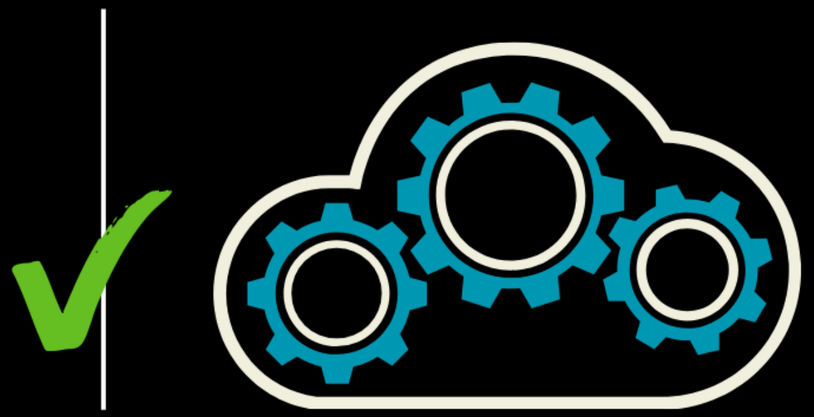
Validator

Fixing Cryptography

Fuzzer



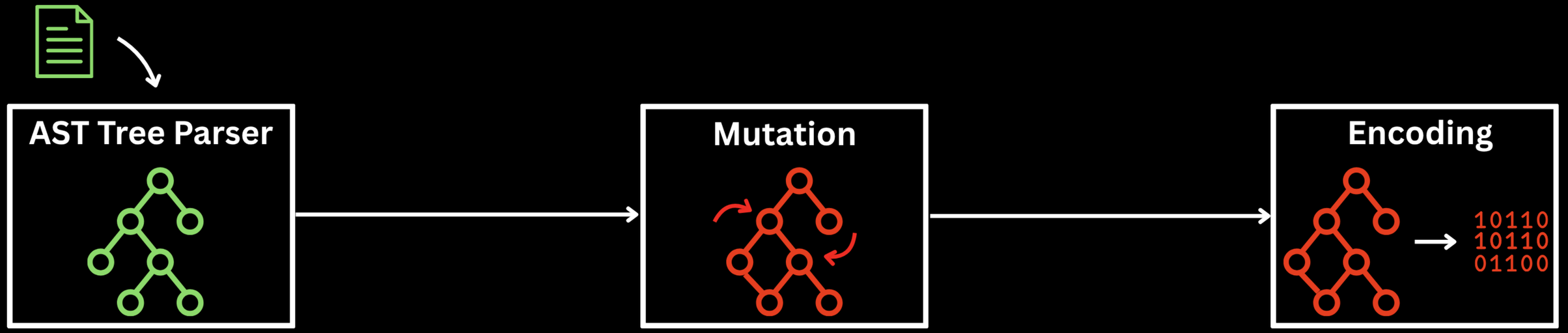
Validator



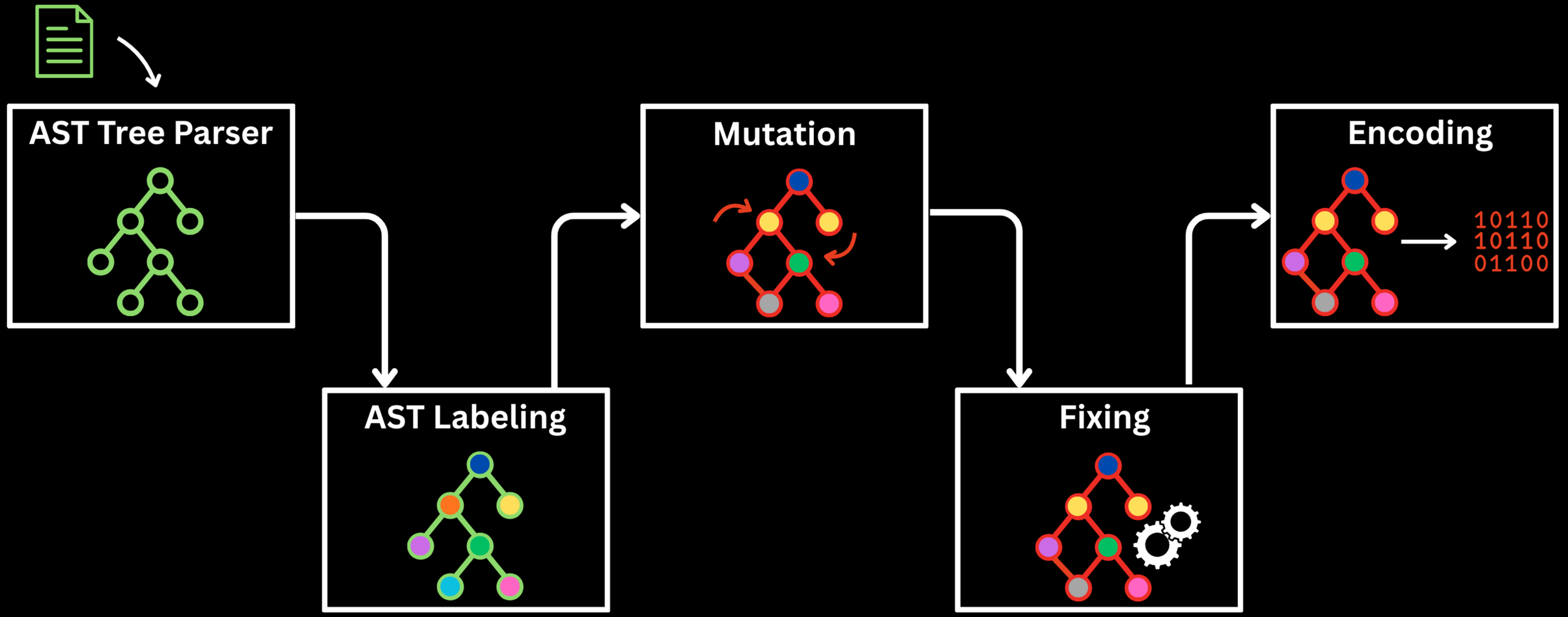
3 - Building the Prototype

An RPKI validator Fuzzer

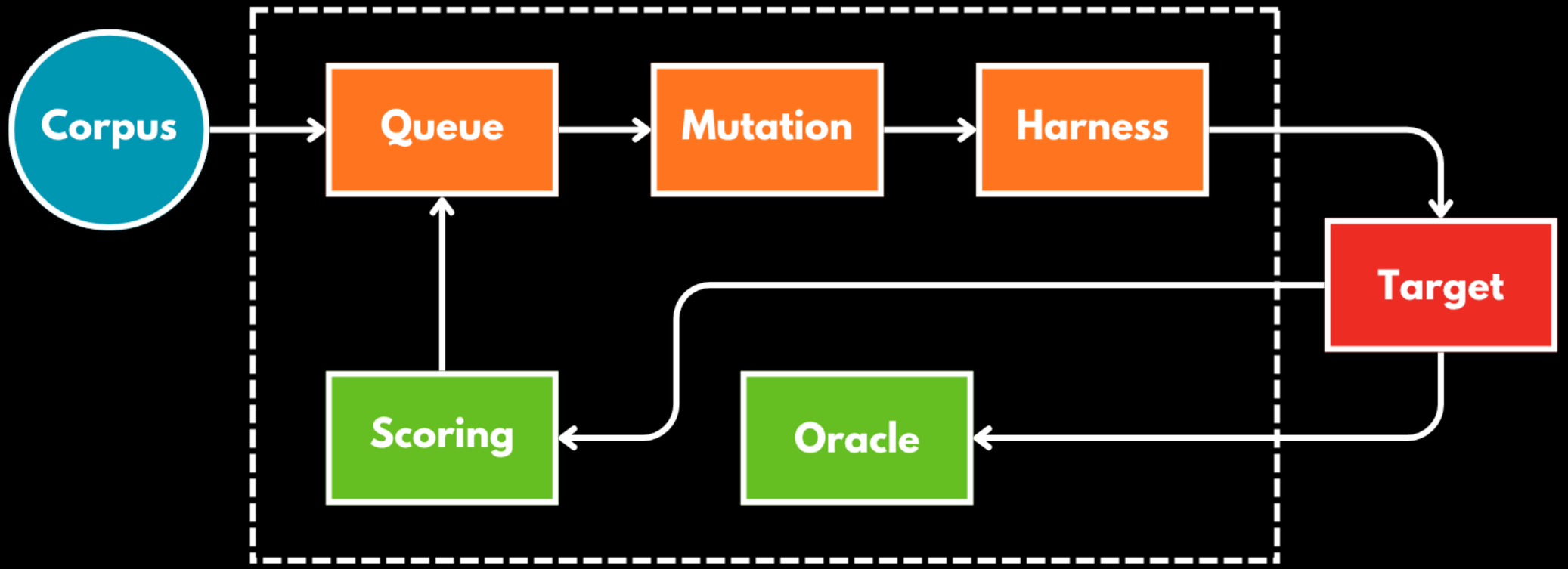
Object Parsing and Mutation



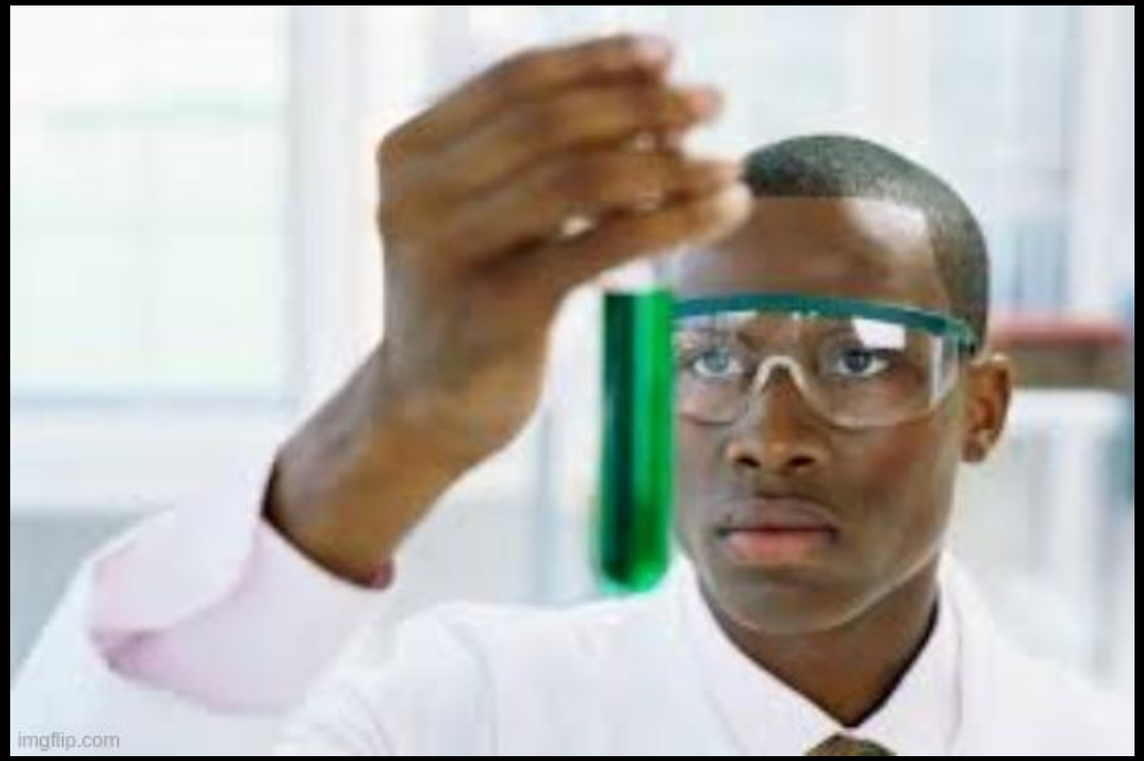
Object Parsing and Mutation



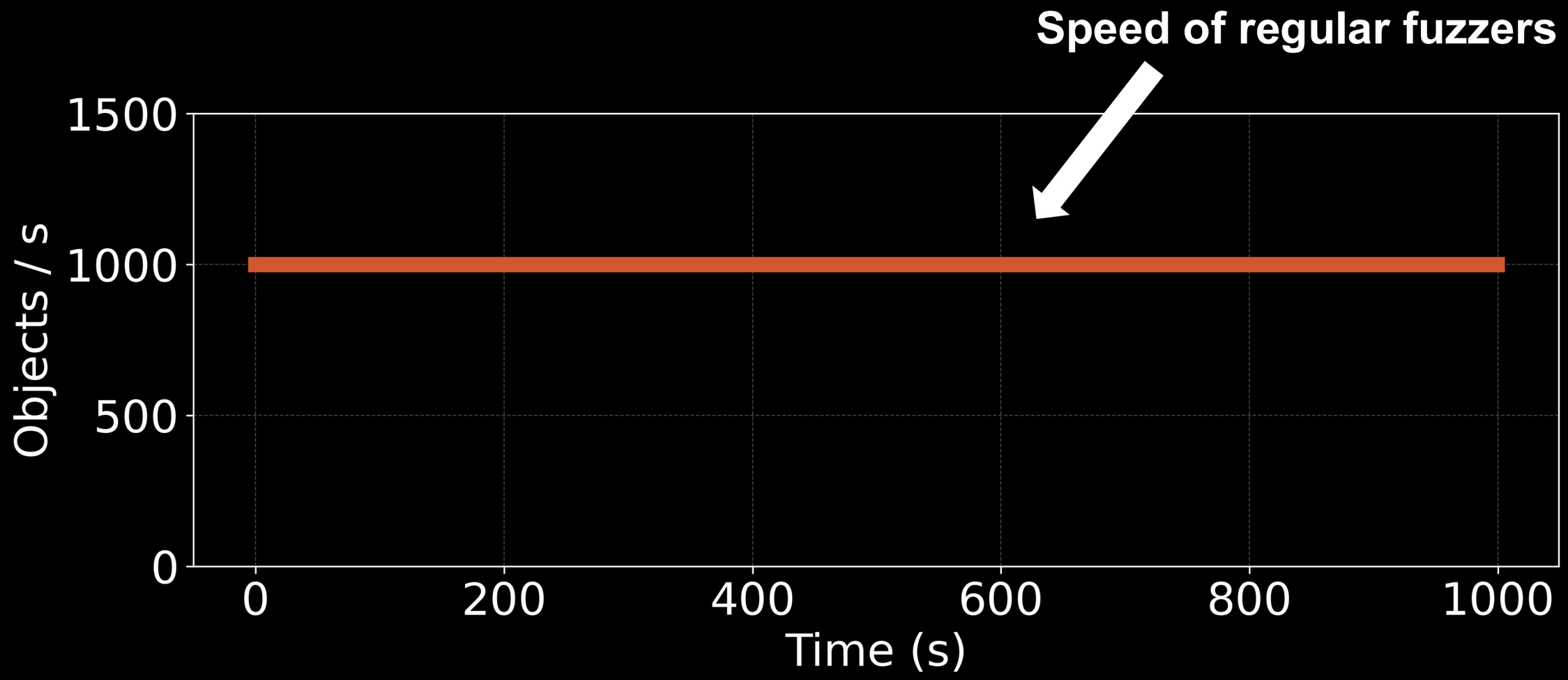
Fuzzing Architecture



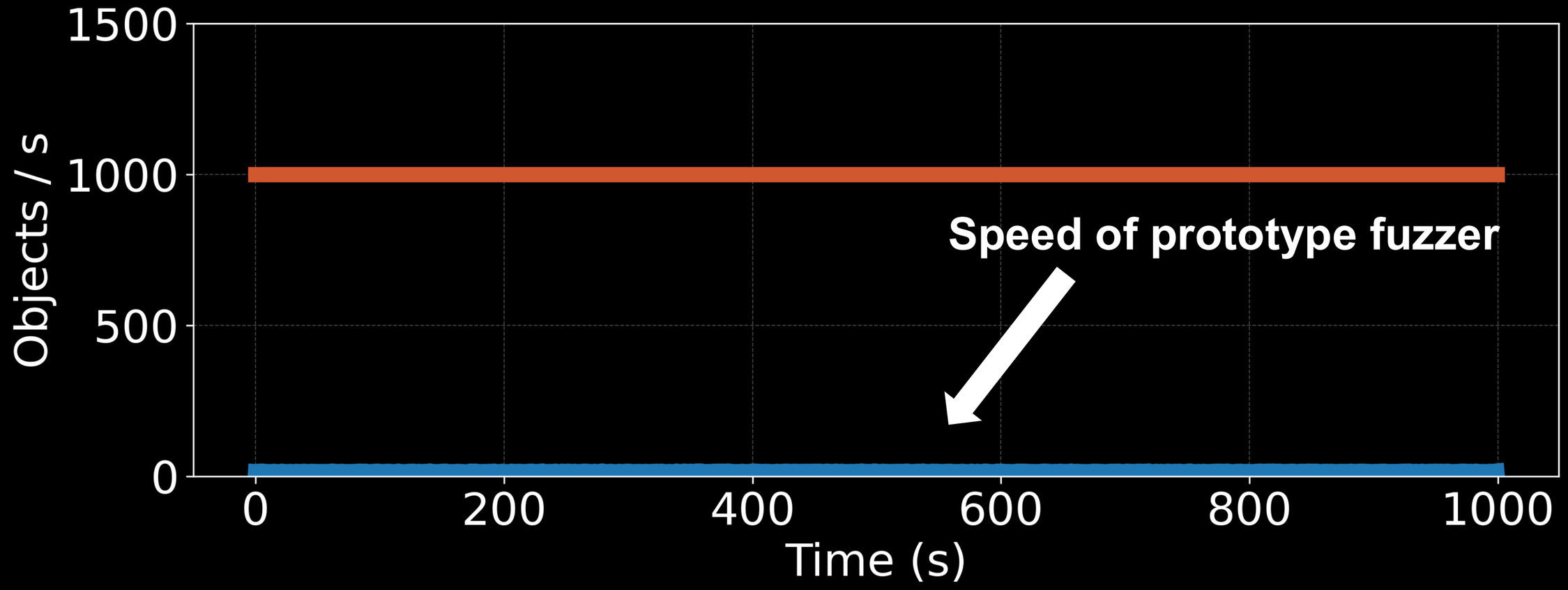
Does this work?



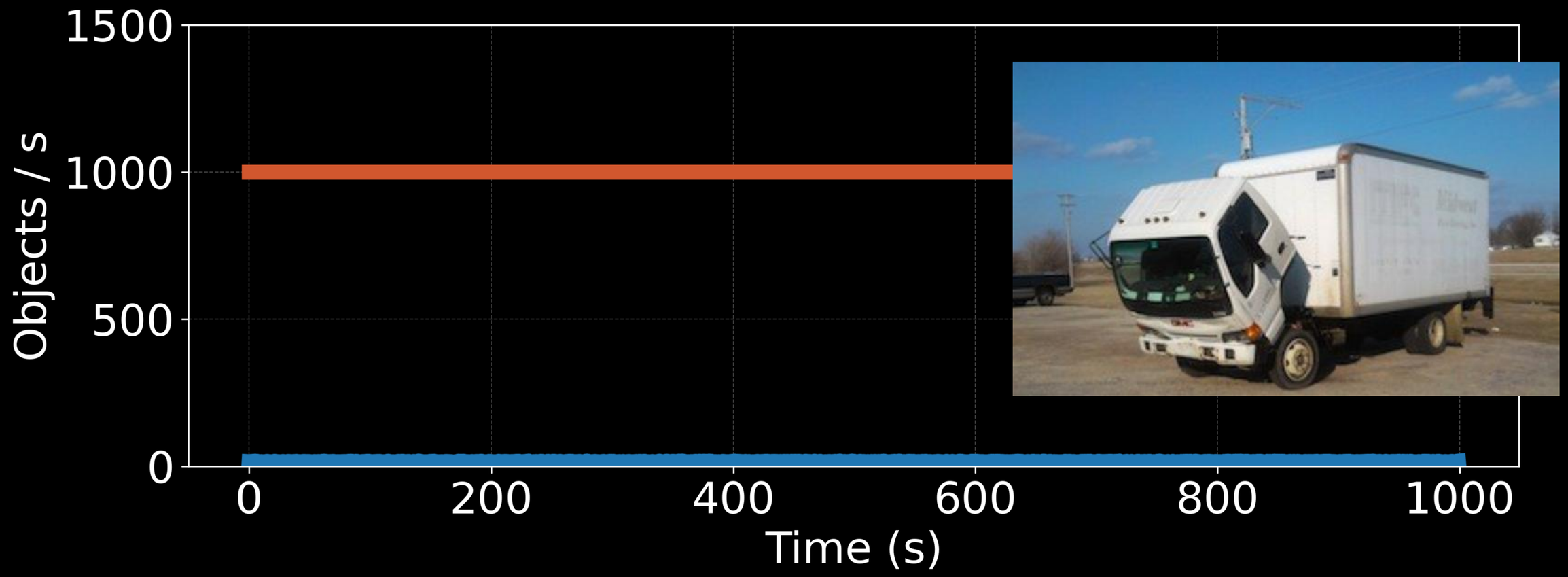
Speed of our Fuzzer



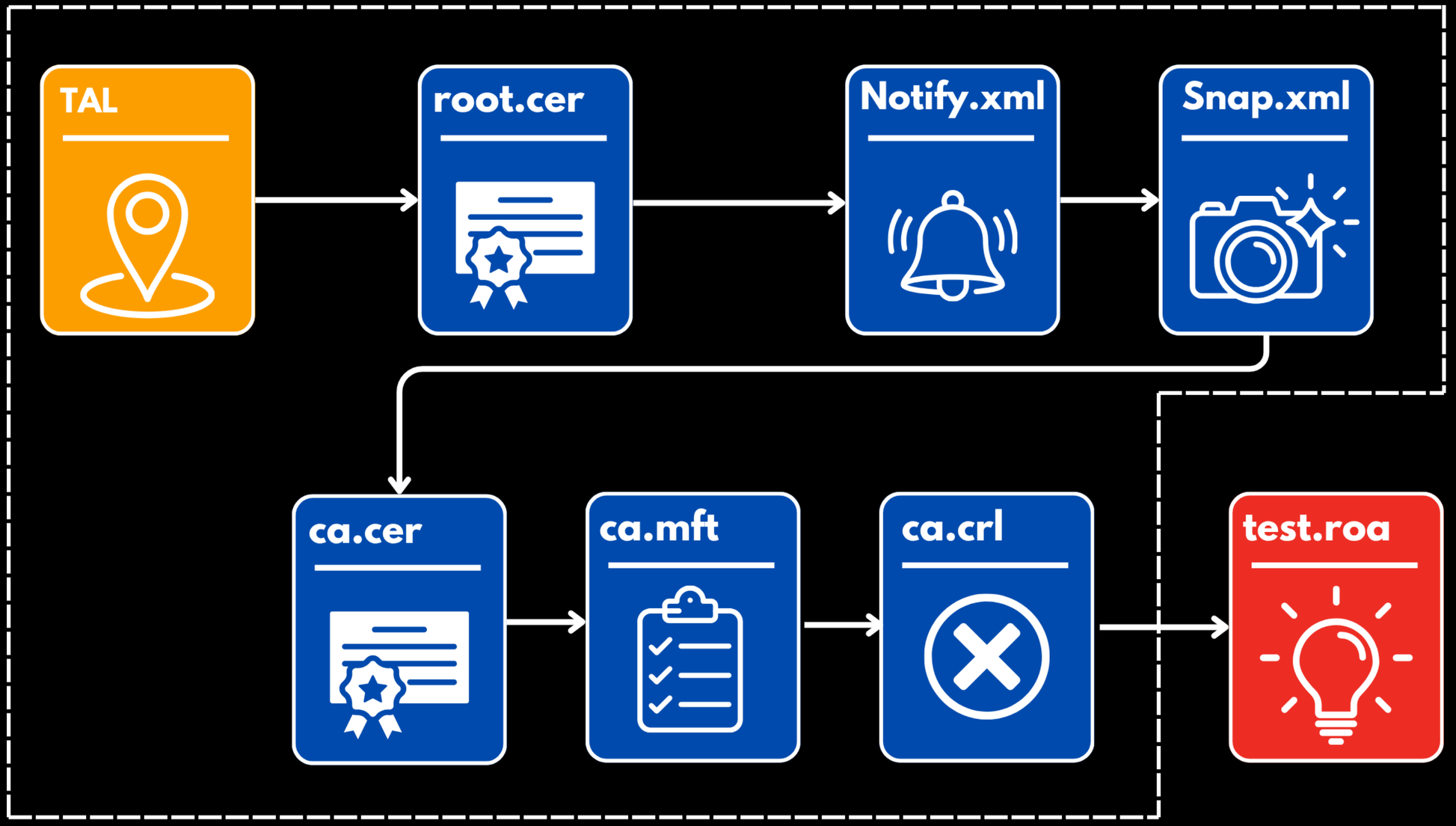
Speed of our Fuzzer



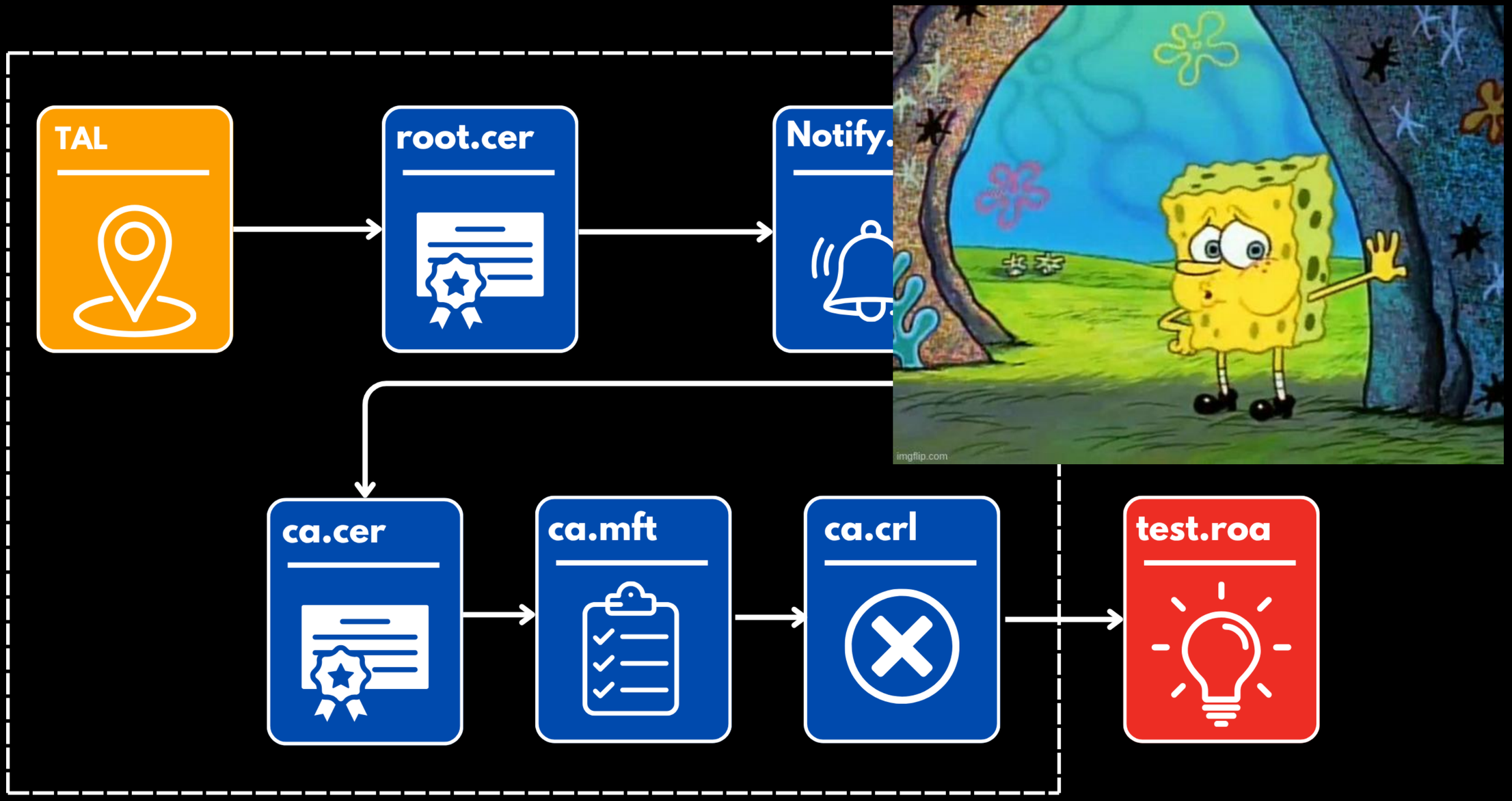
Speed of our Fuzzer



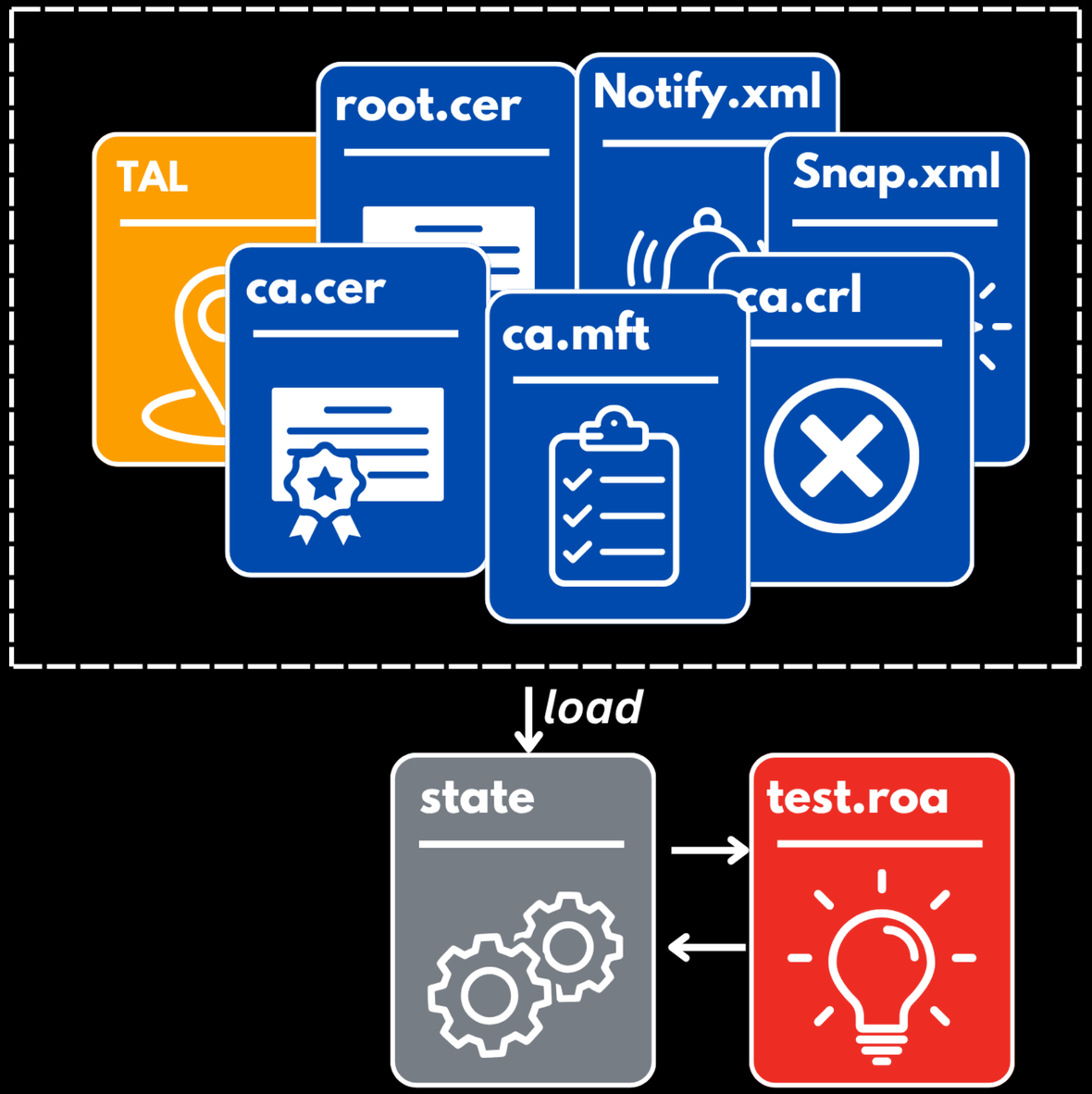
The challenge of fuzzing RPKI



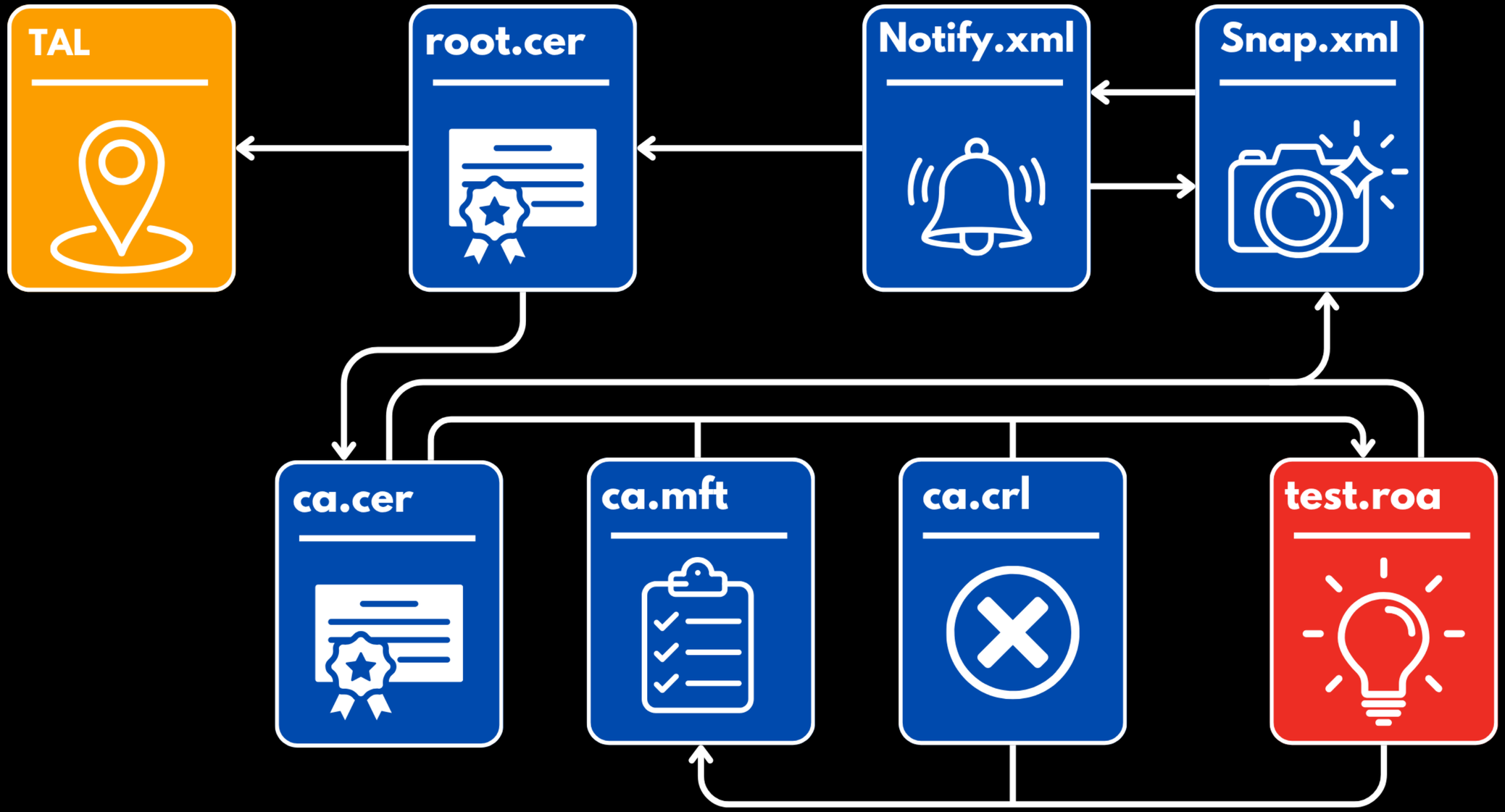
The challenge of fuzzing RPKI



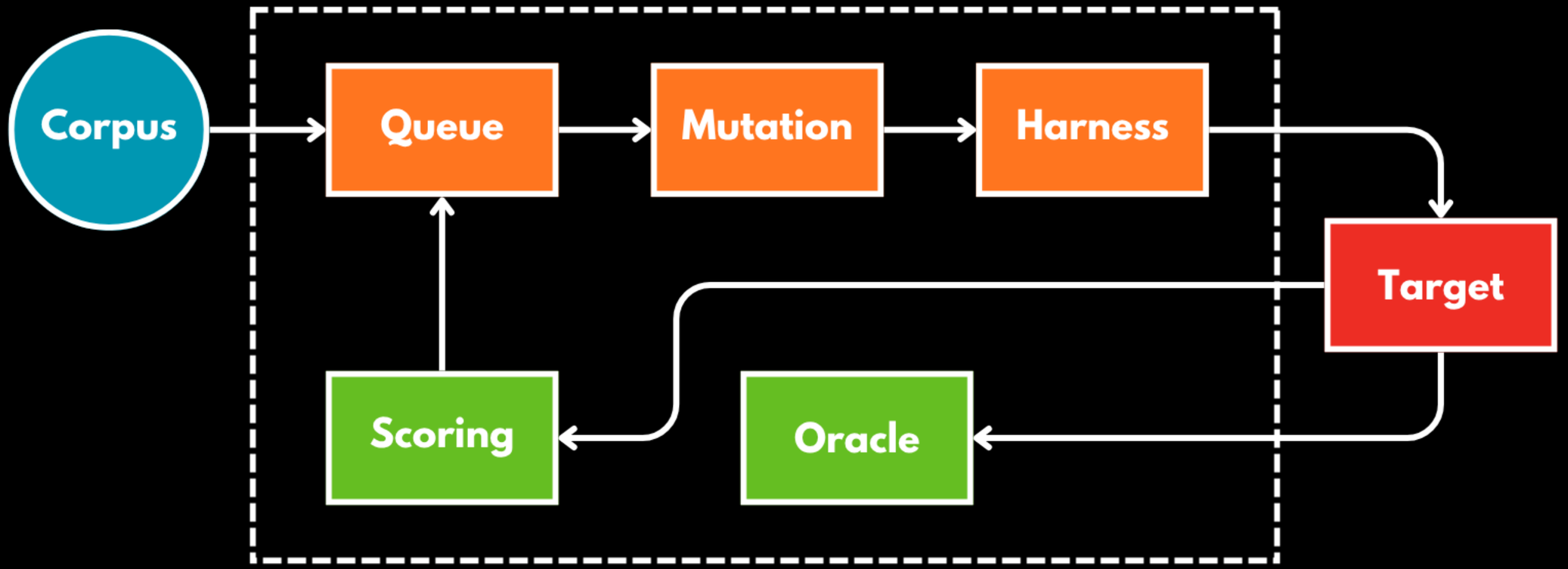
Snapshotting Setup



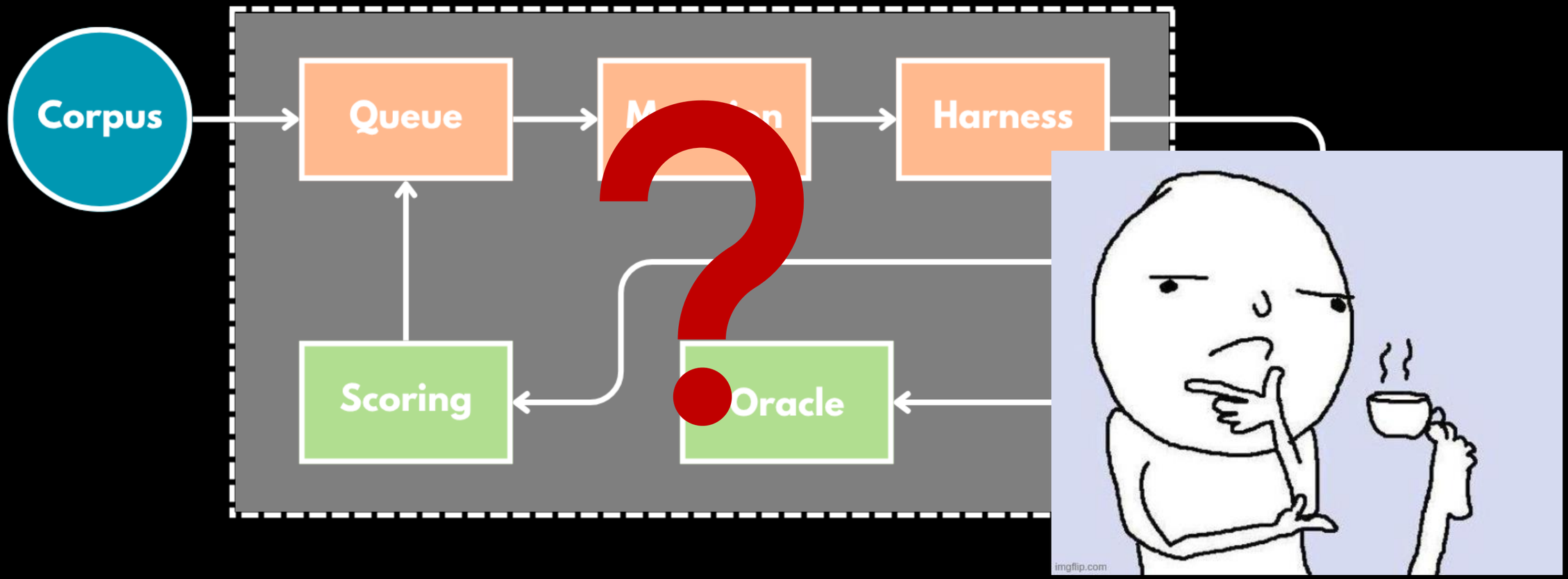
Snapshotting doesn't work



Sequential Fuzzing Architecture



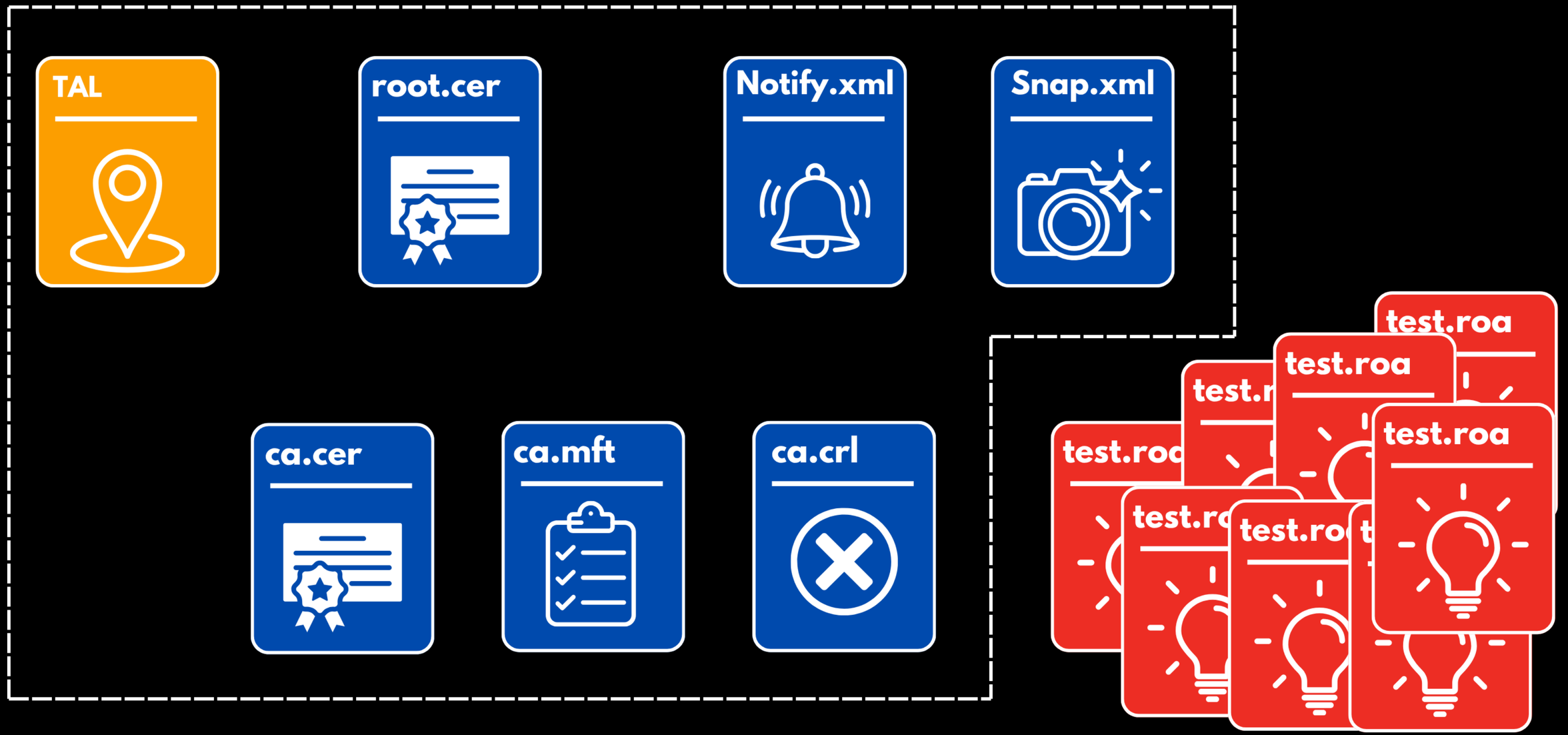
Sequential Fuzzing Architecture



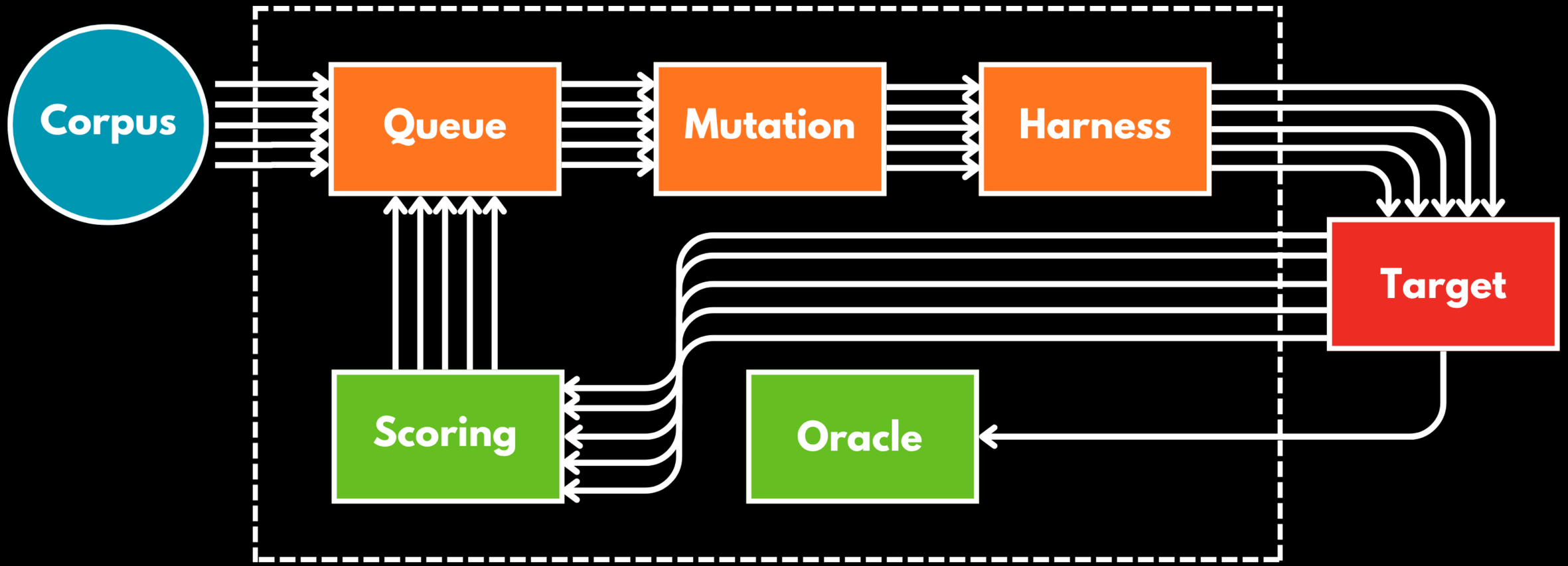
4 - Making it Powerful

A new fuzzing Architecture

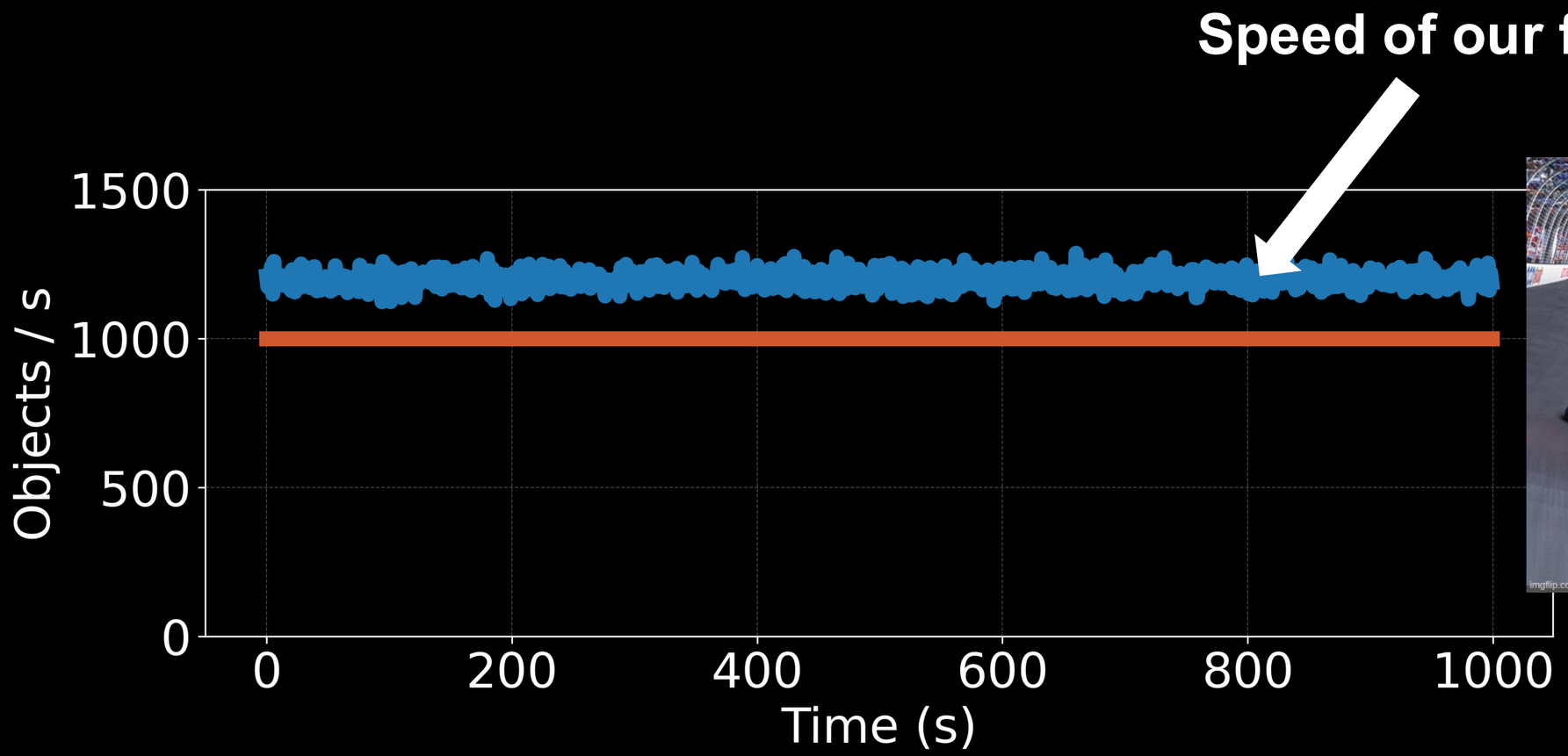
Batching Inputs

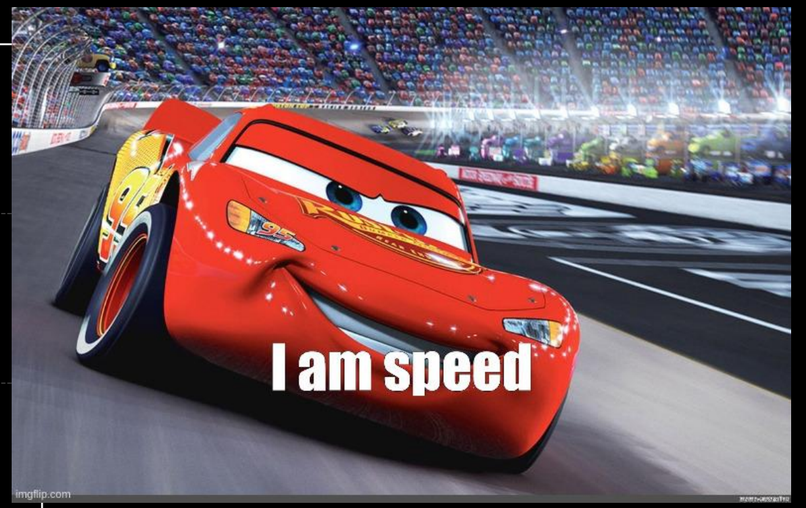


Batched (parallel) Fuzzing

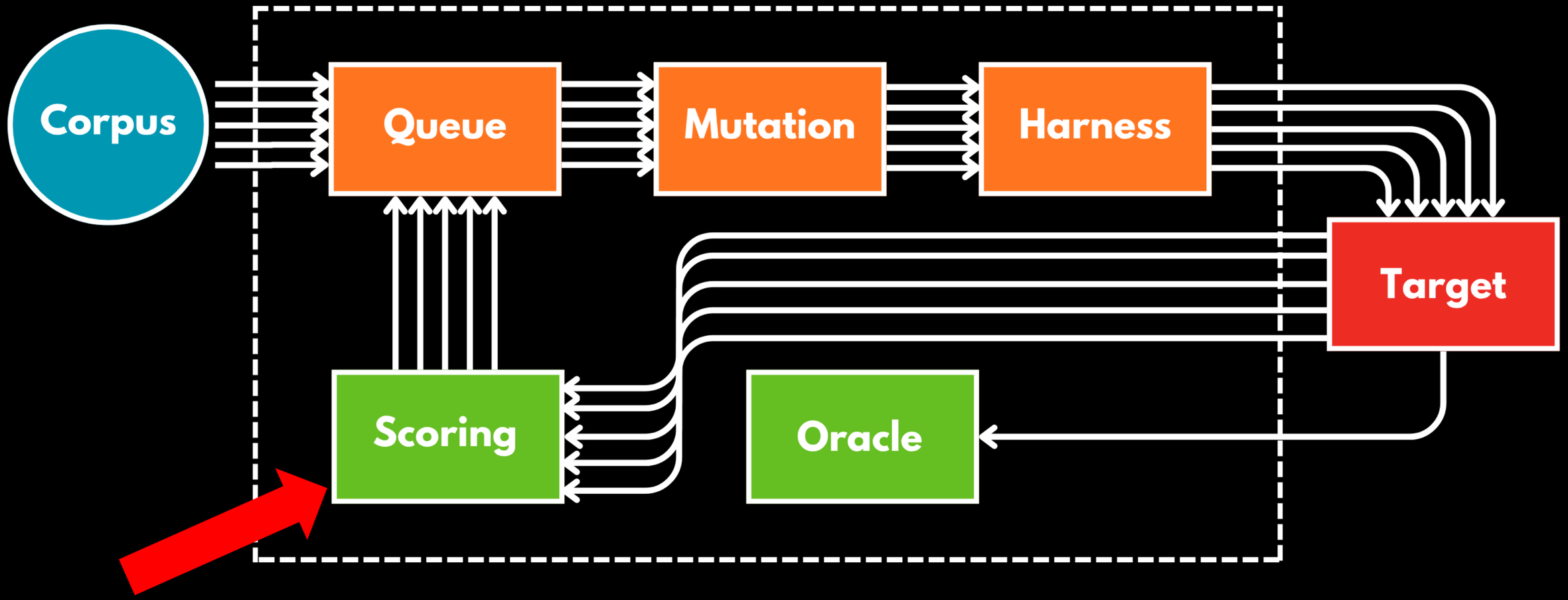


Batched (parallel) Fuzzing

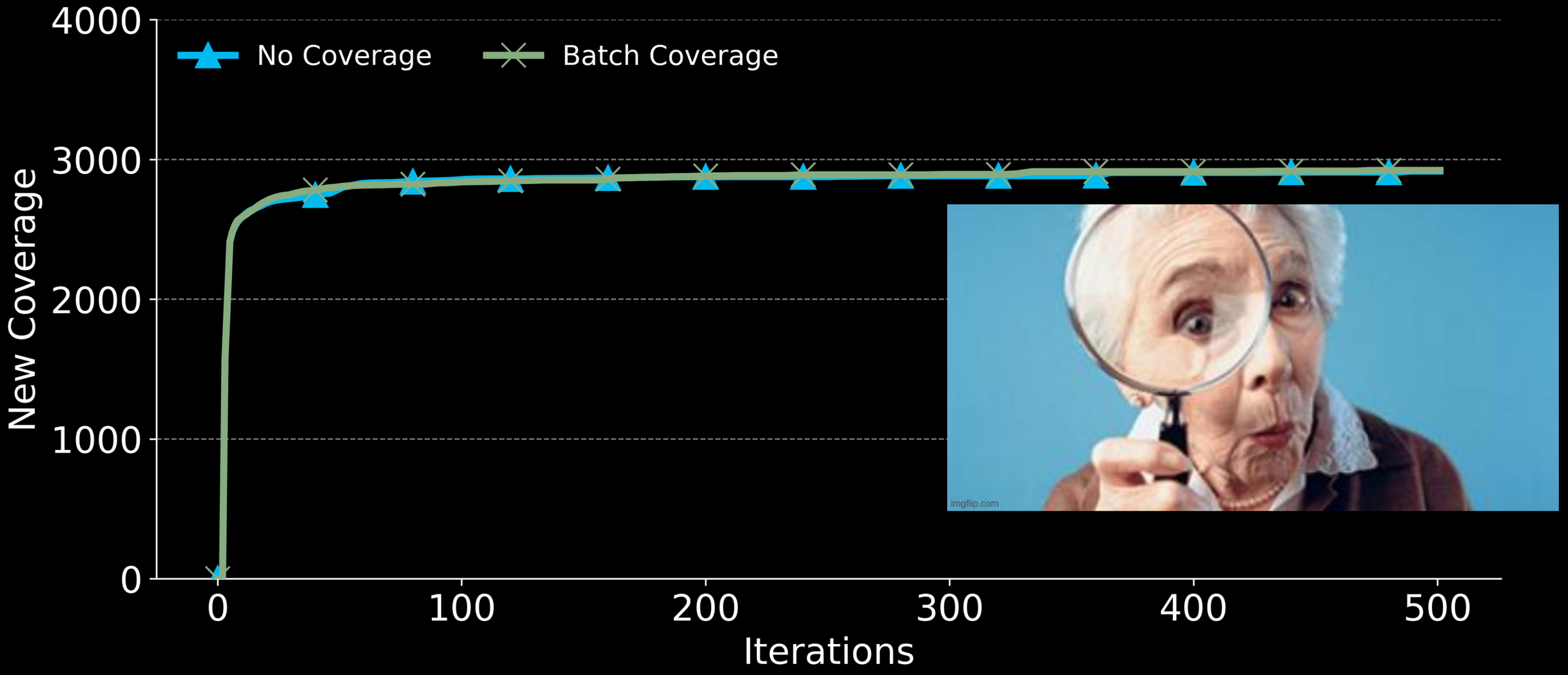




Batched (parallel) Fuzzing



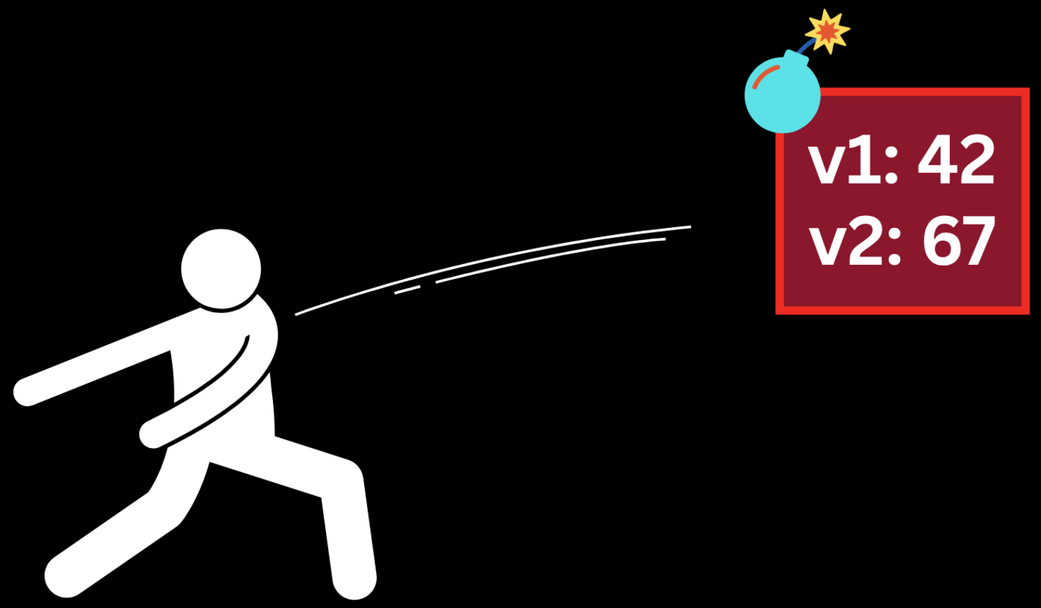
Fuzzing in batches loses coverage benefit



Coverage-guided Fuzzing

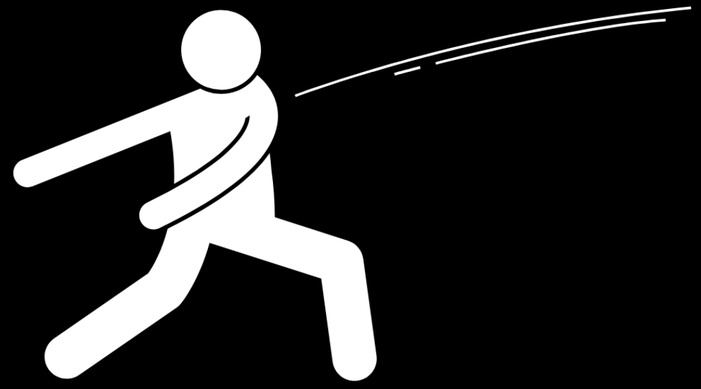
```c
void test(int v1, int v2, char *b, size_t b_len) {
    char buf[32];

    if (v1 == 42) {
        if (v2 = 21){
            memcpy(buf, b, b_len); // overflow
        }
    }
}
```

Coverage-guided Fuzzing



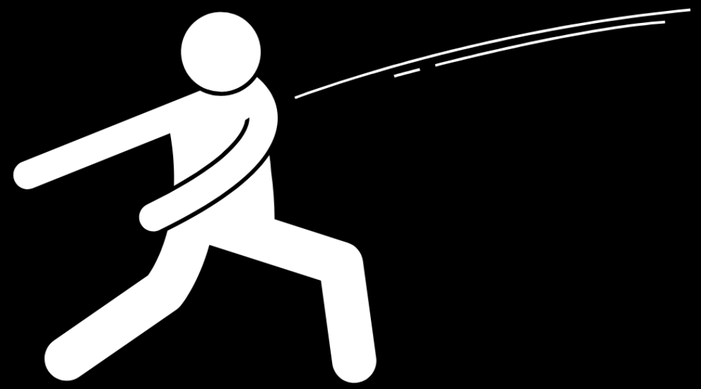

```
void test(int v1, int v2, char *b, size_t b_len) {
    char buf[32];

    if (v1 == 42) {
        if (v2 = 21){
            memcpy(buf, b, b_len); // overflow
        }
    }
}
```

Coverage-guided Fuzzing




```
void test(int v1, int v2, char *b, size_t b_len) {
    char buf[32];

    if (v1 == 42) {
        if (v2 = 21){
            memcpy(buf, b, b_len); // overflow
        }
    }
}
```

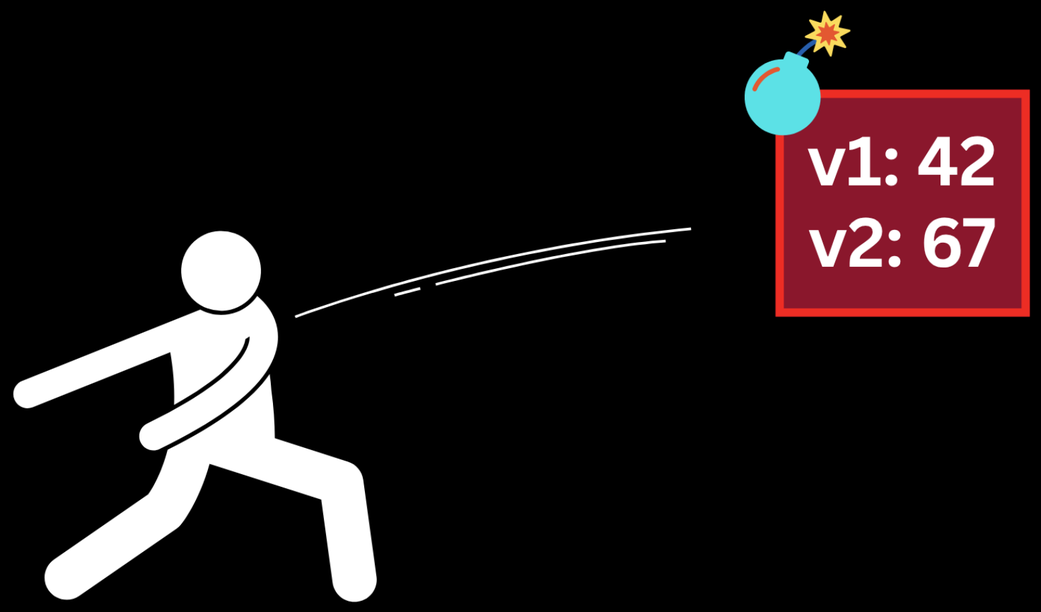
Coverage-guided Fuzzing



```
void test(int v1, int v2, char *b, size_t b_len) {
    // cov_counter_1++

    char buf[32];

    if (v1 == 42) {
        // cov_counter_2++

        if (v2 = 21){
            // cov_counter_3++

            memcpy(buf, b, b_len); // overflow
        }
    }
}
```

cov_counter_1	cov_counter_2	cov_counter_3
0	0	0

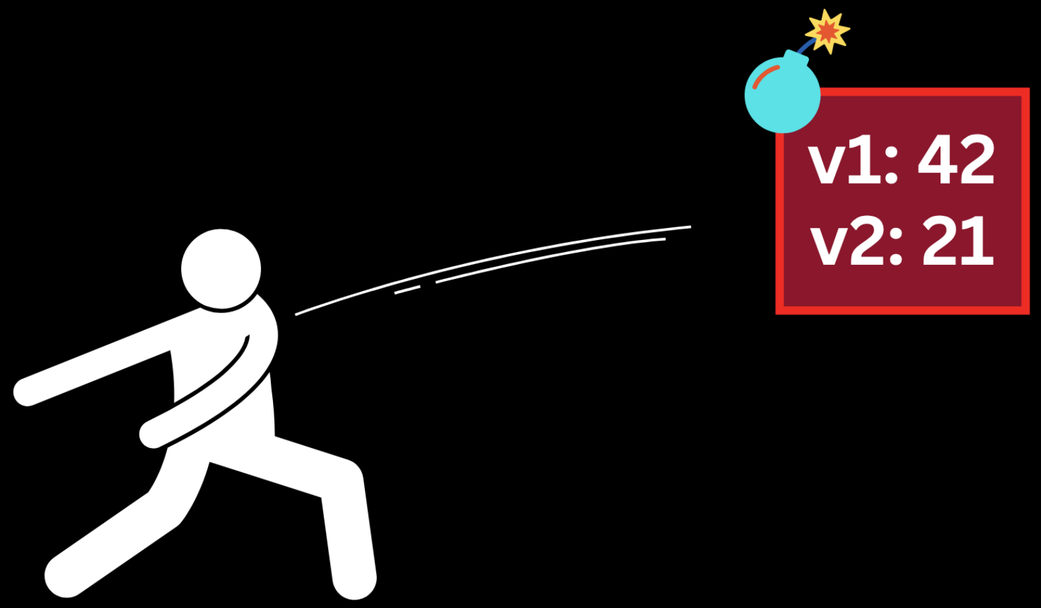
Coverage-guided Fuzzing




```
void test(int v1, int v2, char *b, size_t b_len) {
    // cov_counter_1++

    char buf[32];

    if (v1 == 42) {
        // cov_counter_2++

        if (v2 = 21){
            // cov_counter_3++

            memcpy(buf, b, b_len); // overflow
        }
    }
}
```

cov_counter_1	cov_counter_2	cov_counter_3
1	1	0

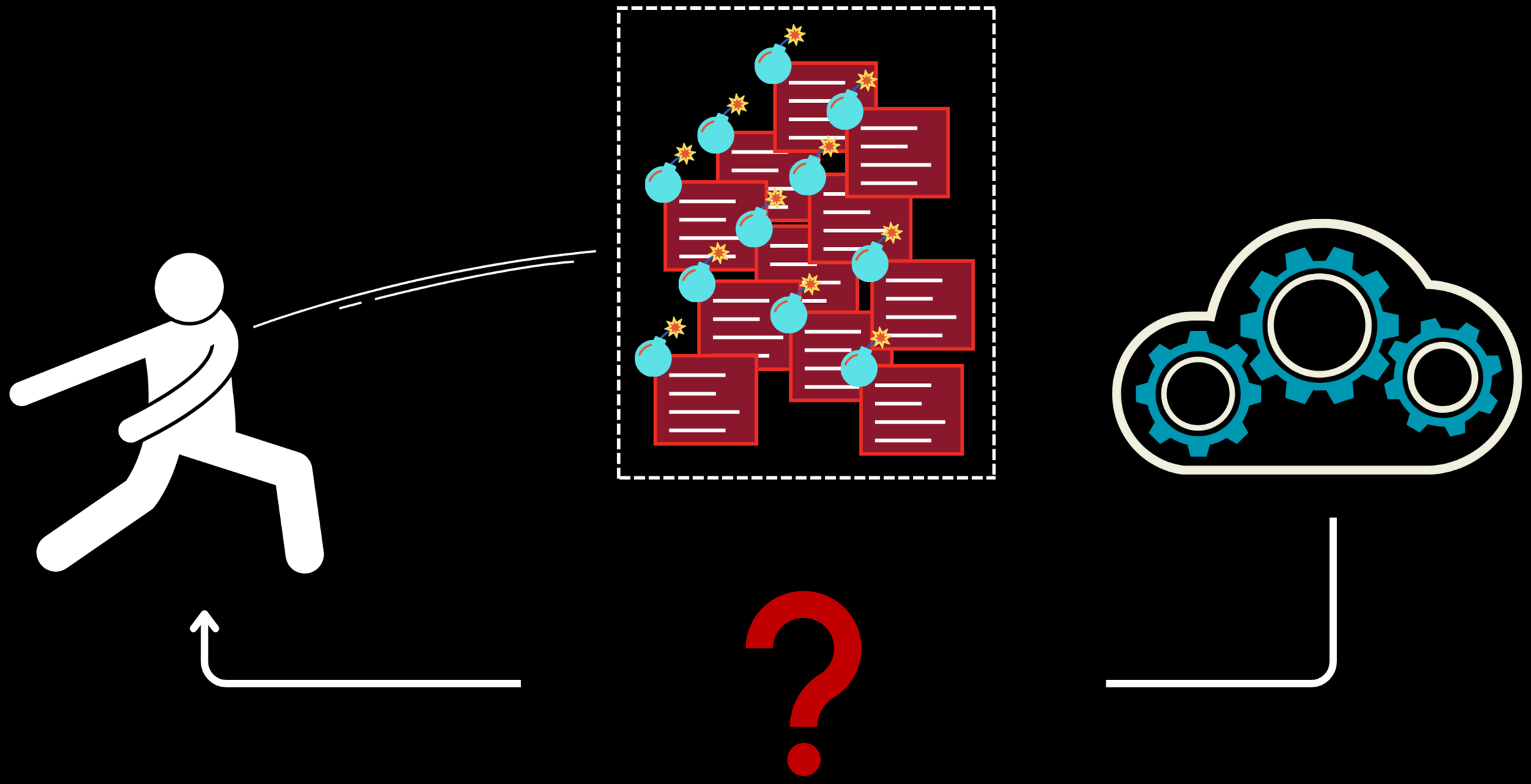
Coverage-guided Fuzzing



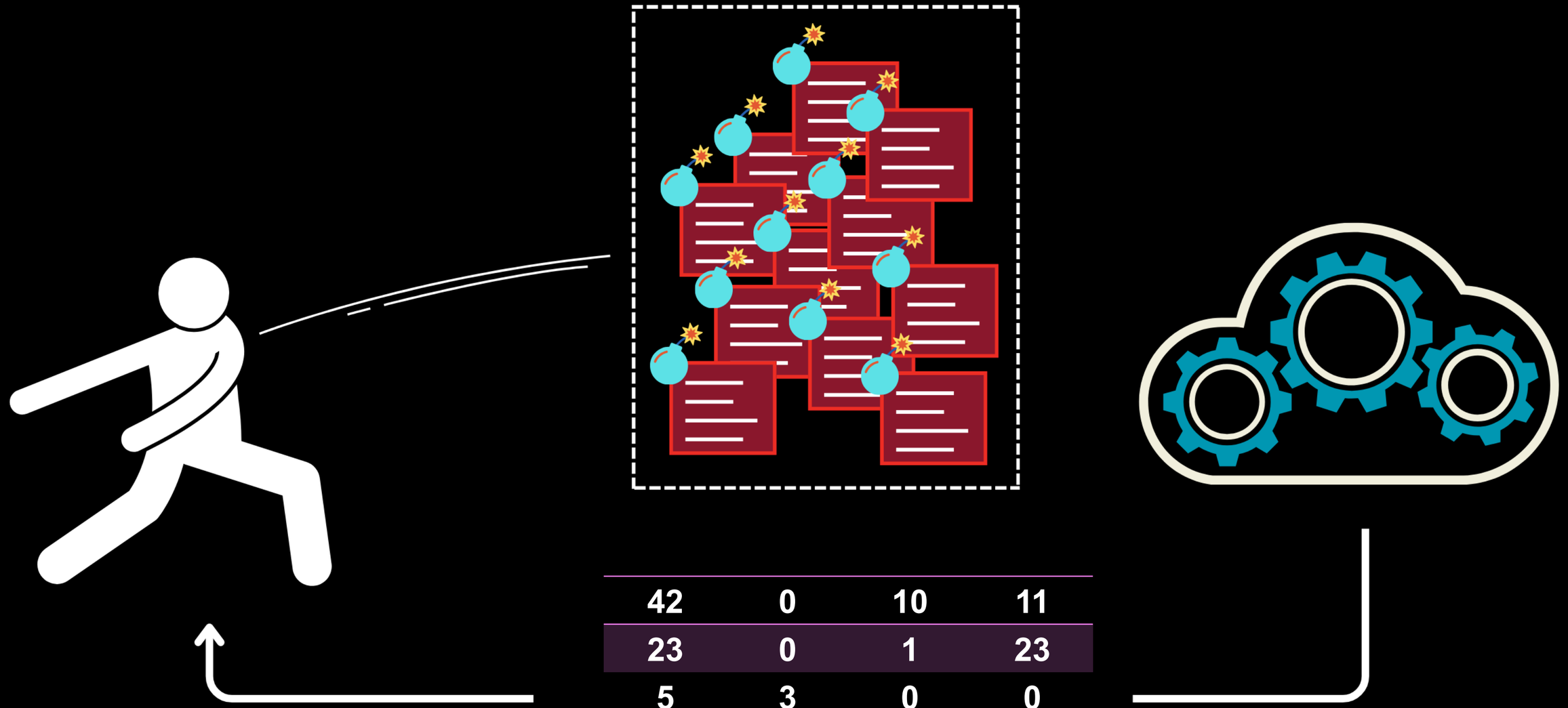

```
void test(int v1, int v2, char *b, size_t b_len) {
    // cov_counter_1++ ✓

    char buf[32];

    if (v1 == 42) {
        // cov_counter_2++ ✓

        if (v2 = 21){
            // cov_counter_3++ ✓

            memcpy(buf, b, b_len); // overflow
        }
    }
}
```

cov_counter_1	cov_counter_2	cov_counter_3
1 ✓	1 ✓	1 ✓

Coverage vs. Batches

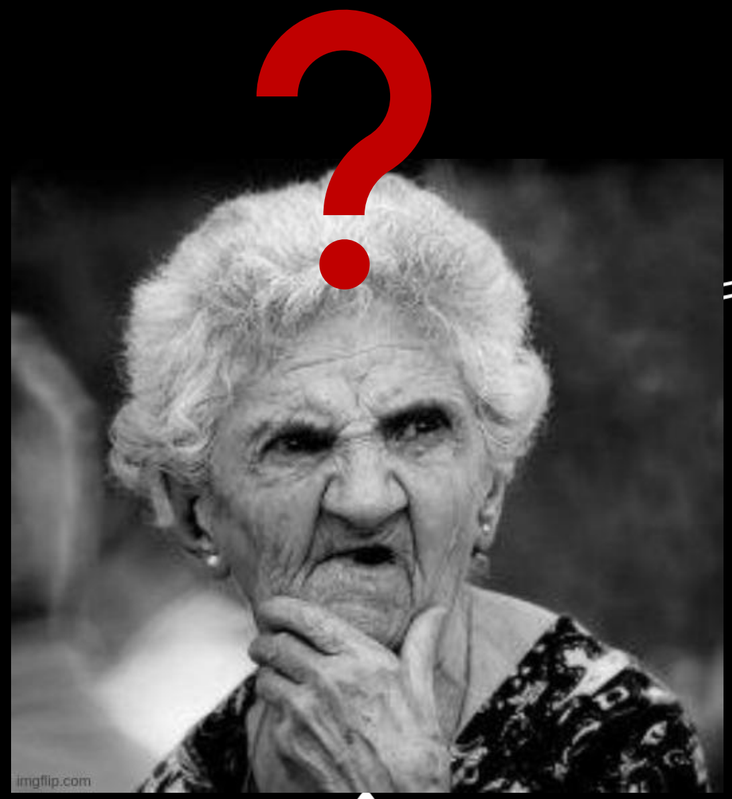


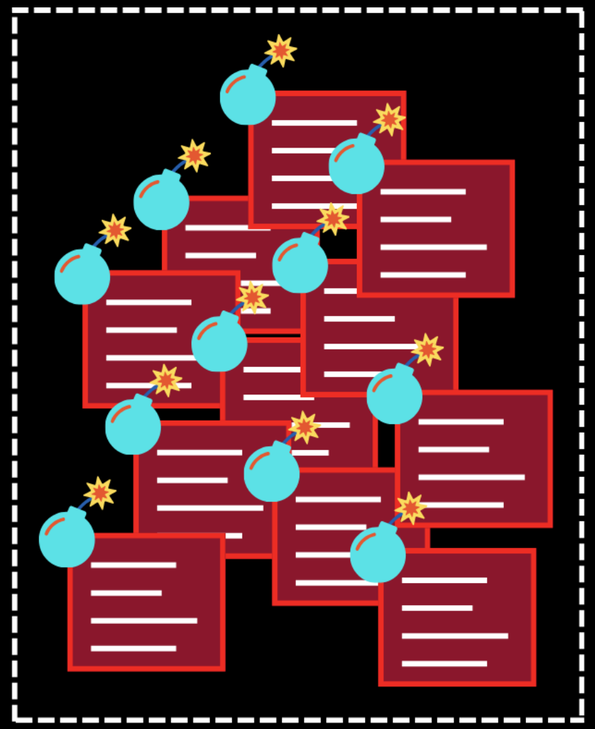
Coverage vs. Batches

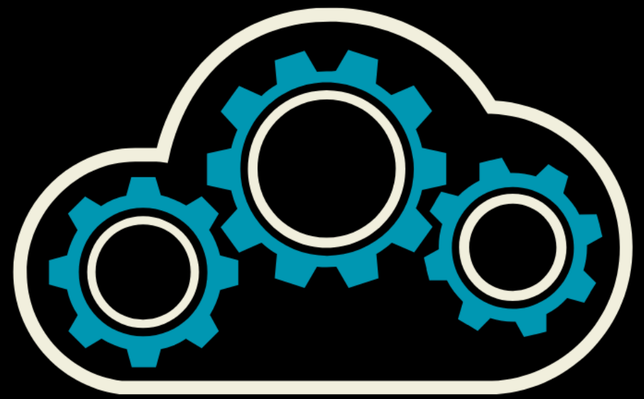


42	0	10	11
23	0	1	23
5	3	0	0
0	32	69	0
3	0	44	1

Coverage vs. Batches

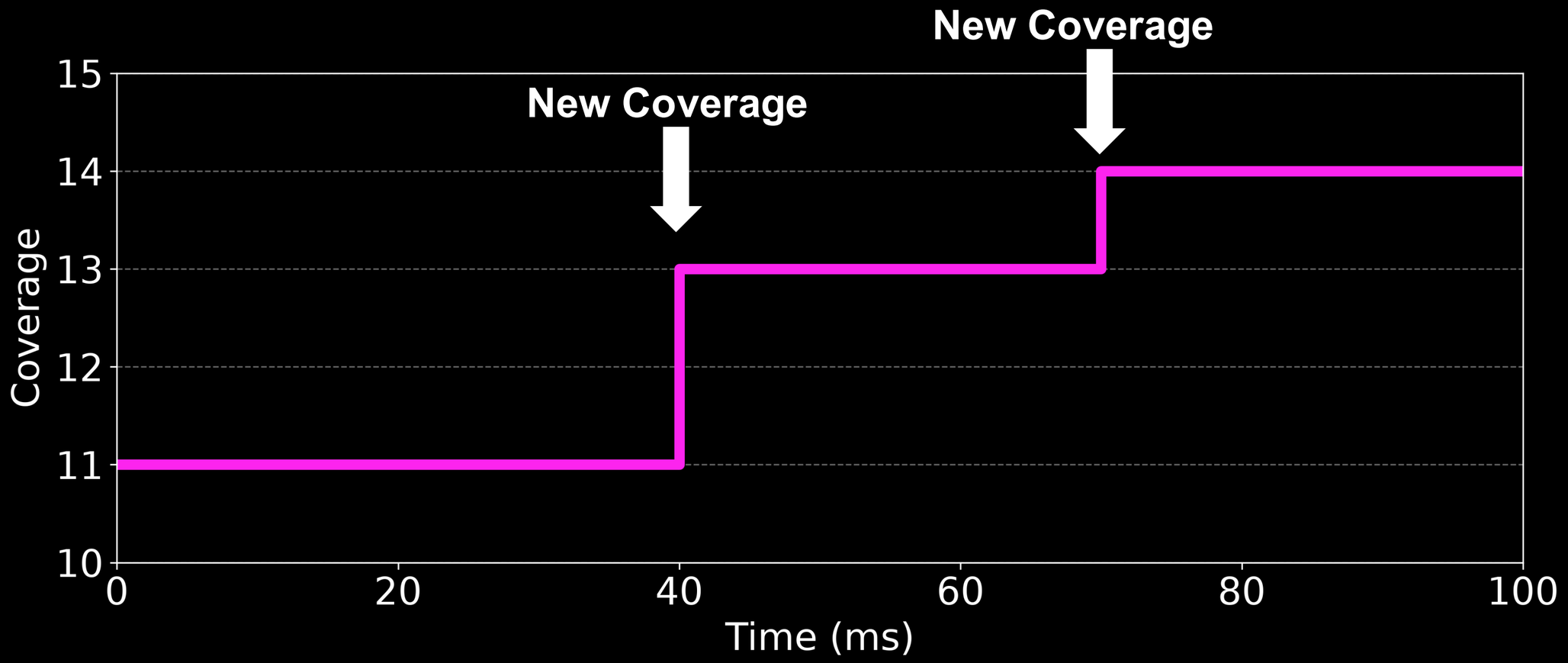




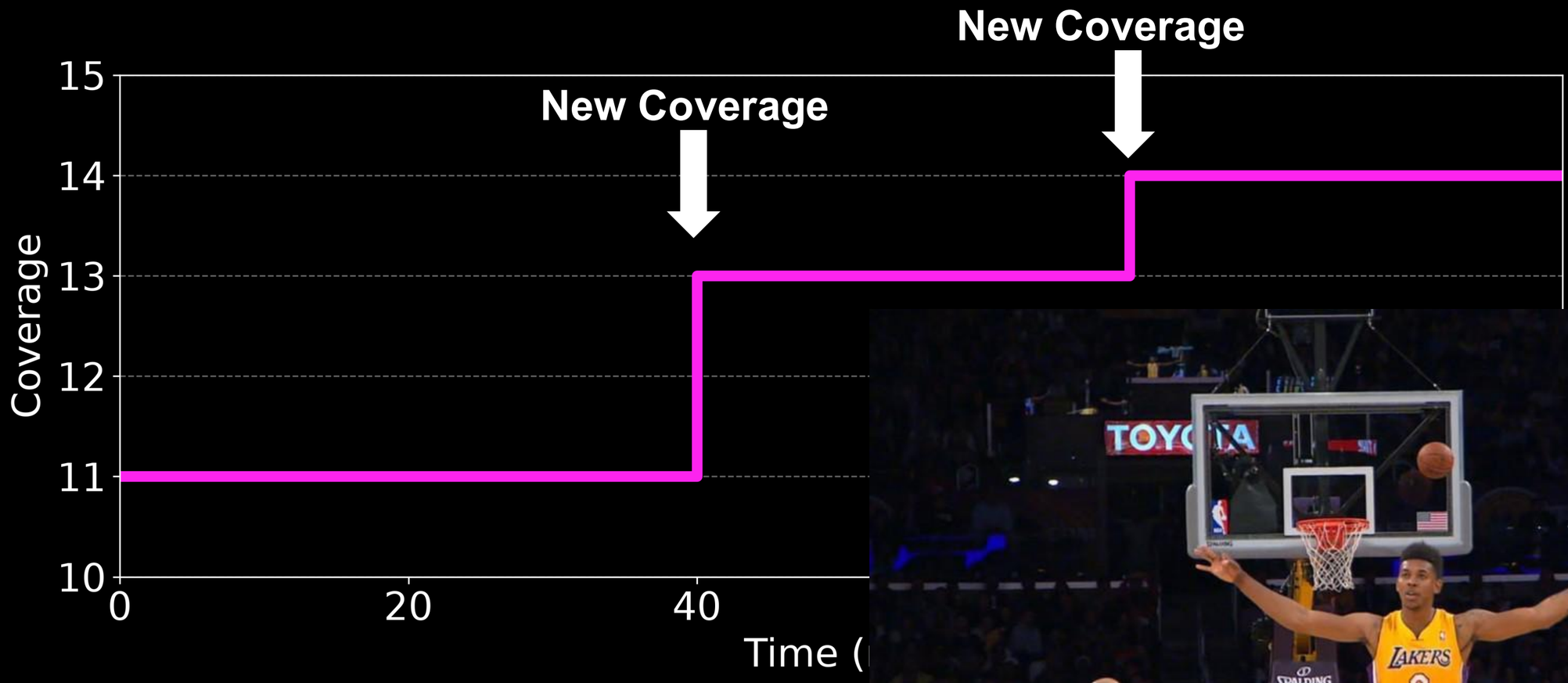


42	0	10	11
23	0	1	23
5	3	0	0
0	32	69	0
3	0	44	1

Idea: Coverage Progression

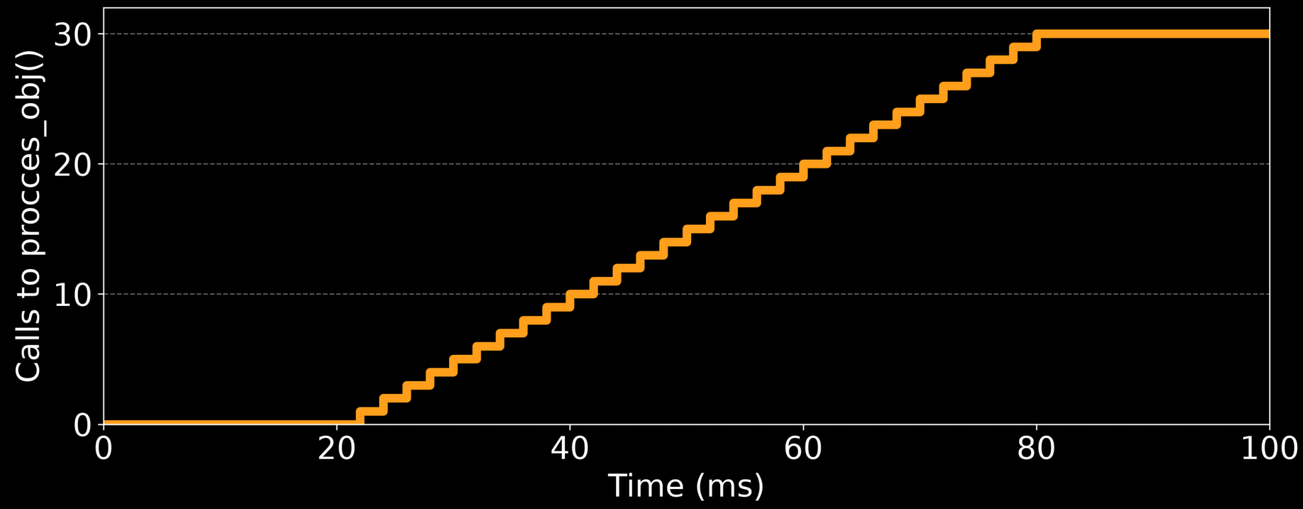


Idea: Coverage Progression



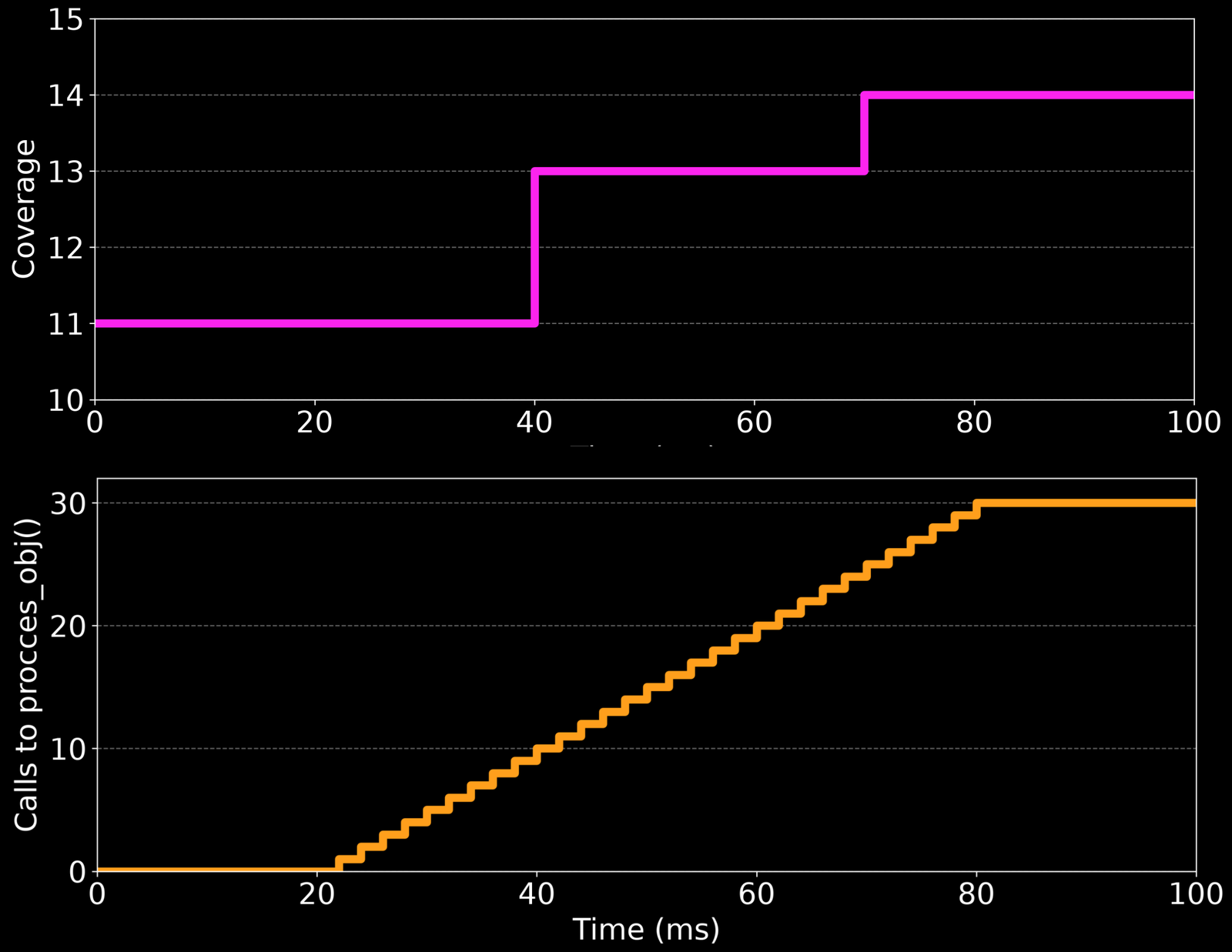
Idea: Coverage Progression

```
for(int i = 0; i < b_len; i++){
    Object obj = objects[i];
    process_obj(obj);
    // ...
}

void process_obj(Object obj){
    // cov_counter_f++

    obj.validate();
    // ...
}
```

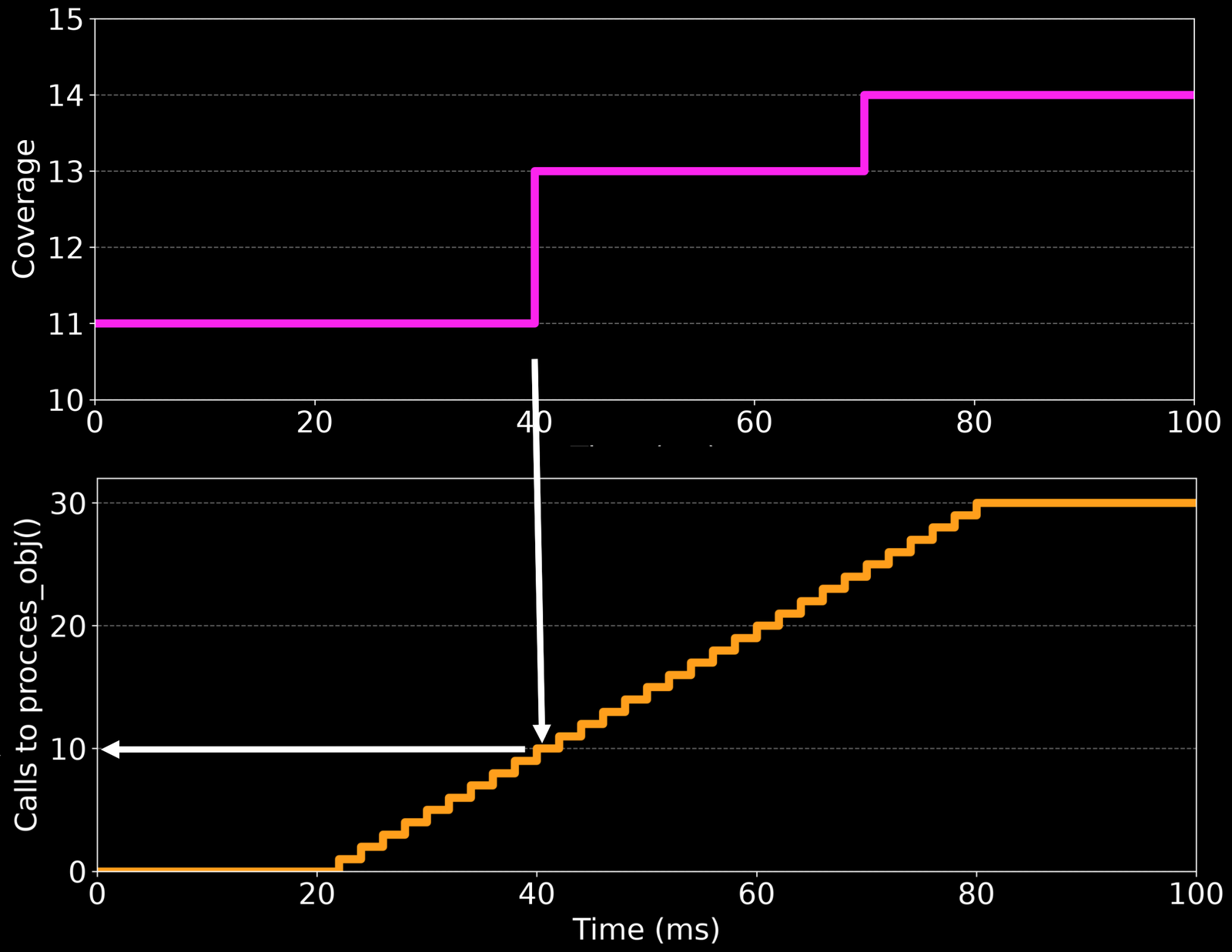


We can use this!

Use functions as timing side-channel

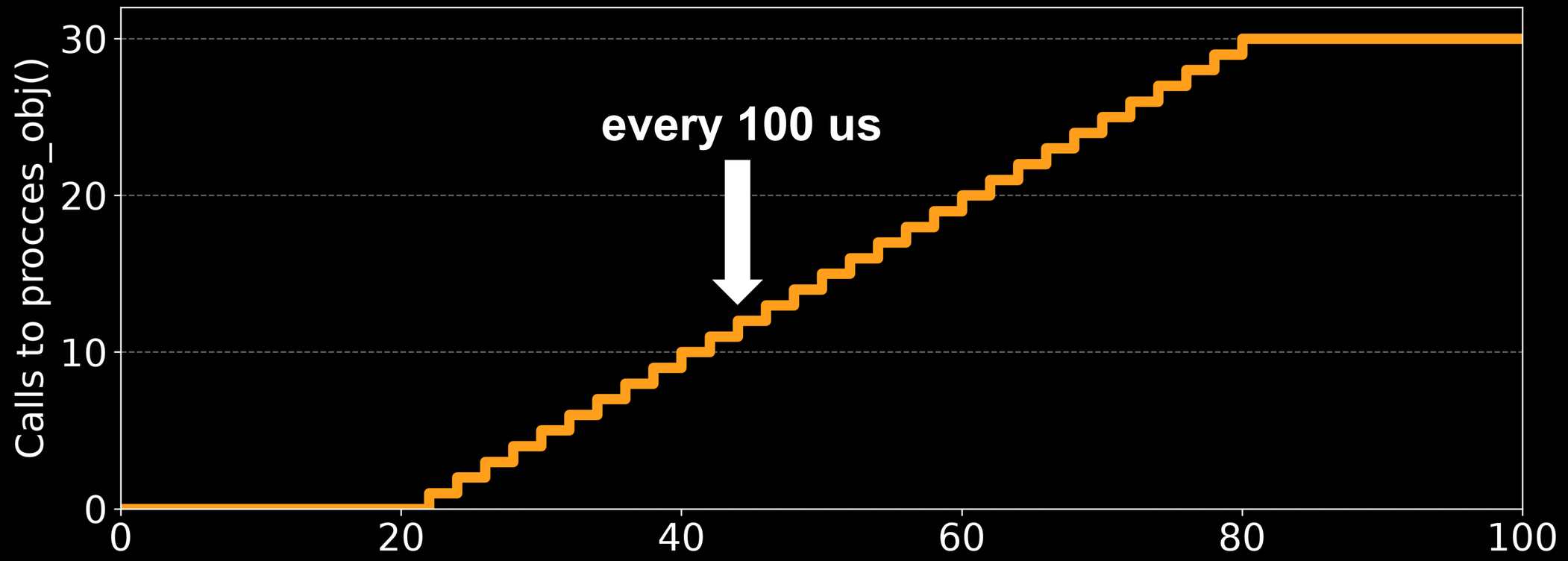


Use functions as timing side-channel

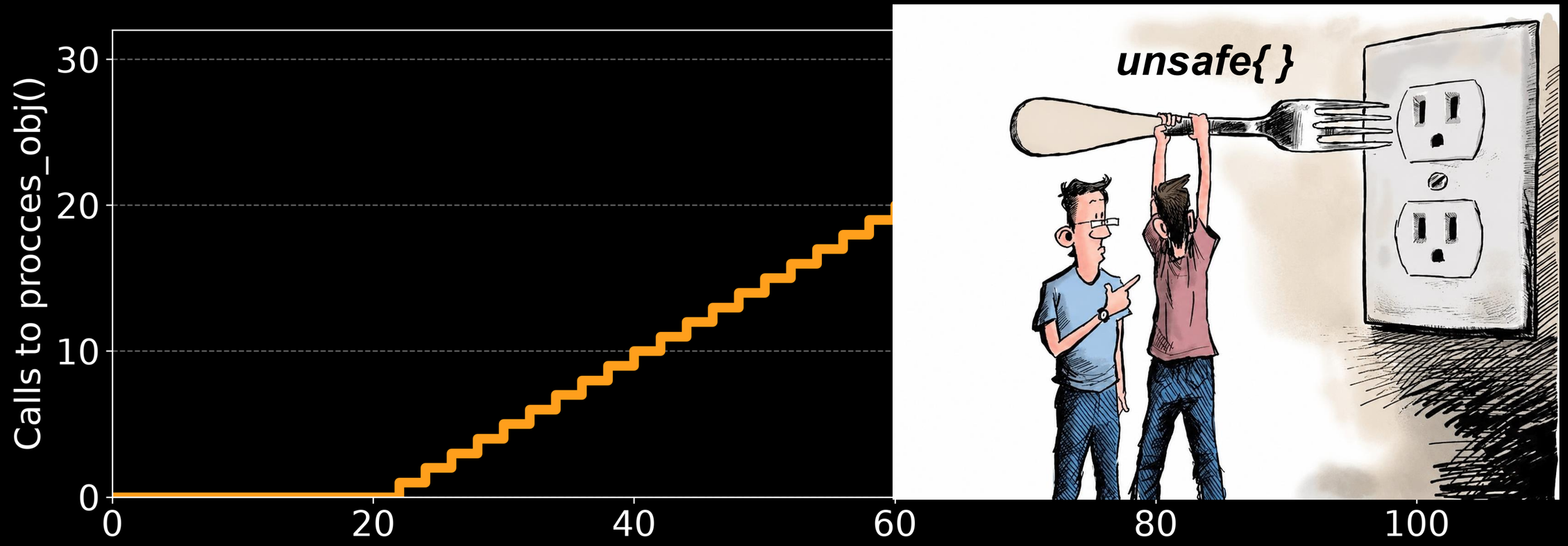


New Coverage

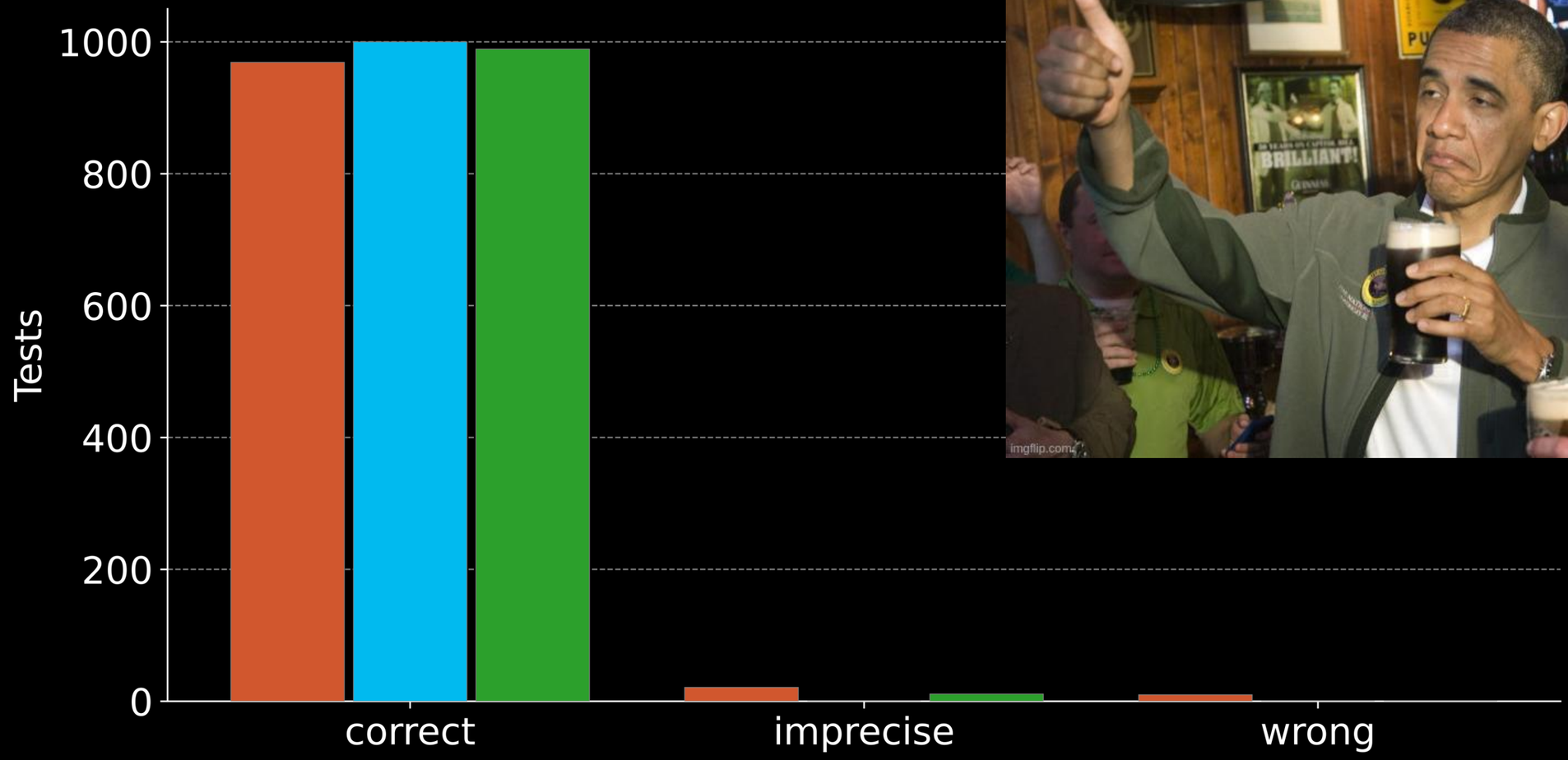
Mapping requires speed



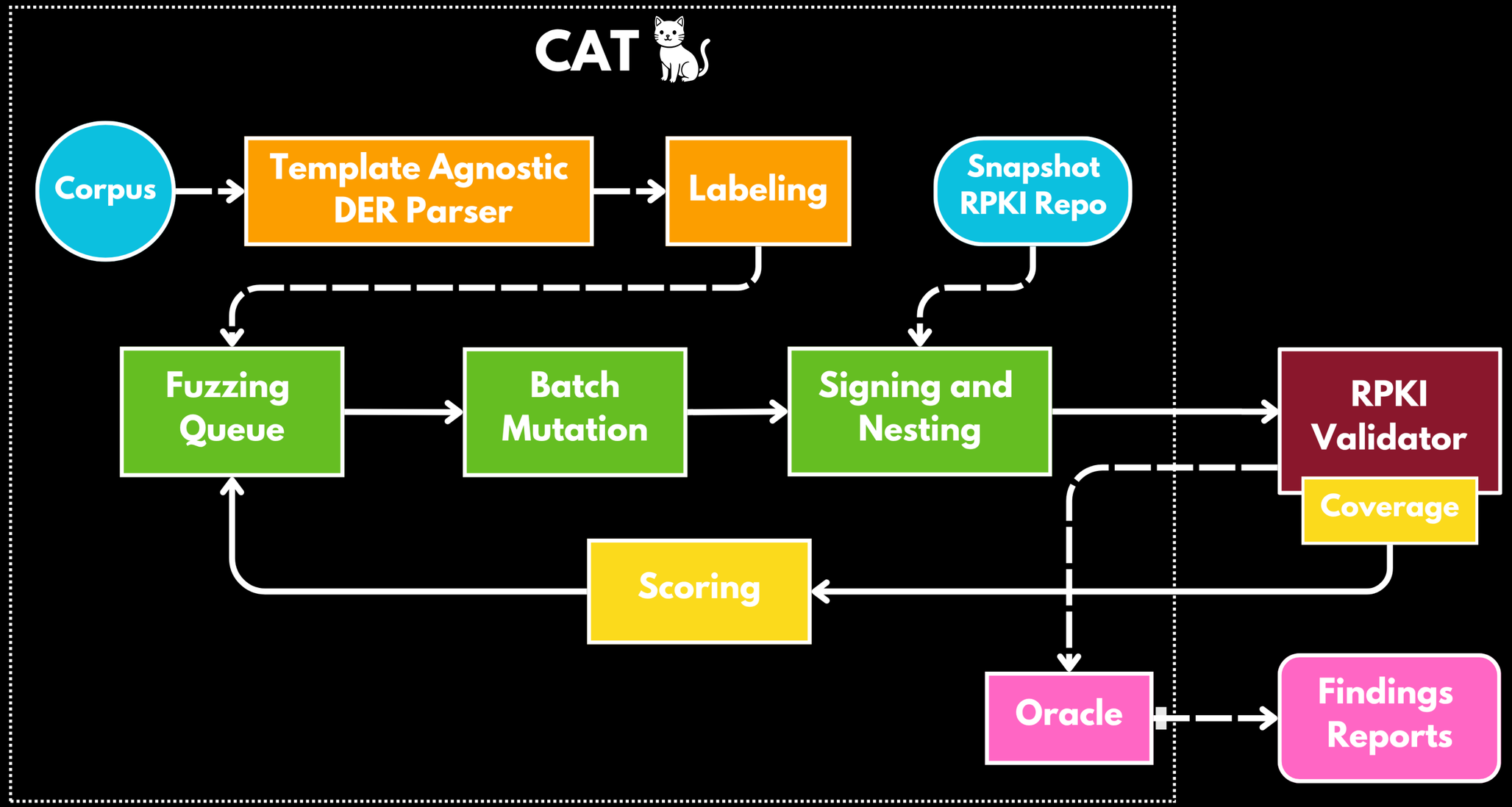
Mapping requires speed



Accurate mapping



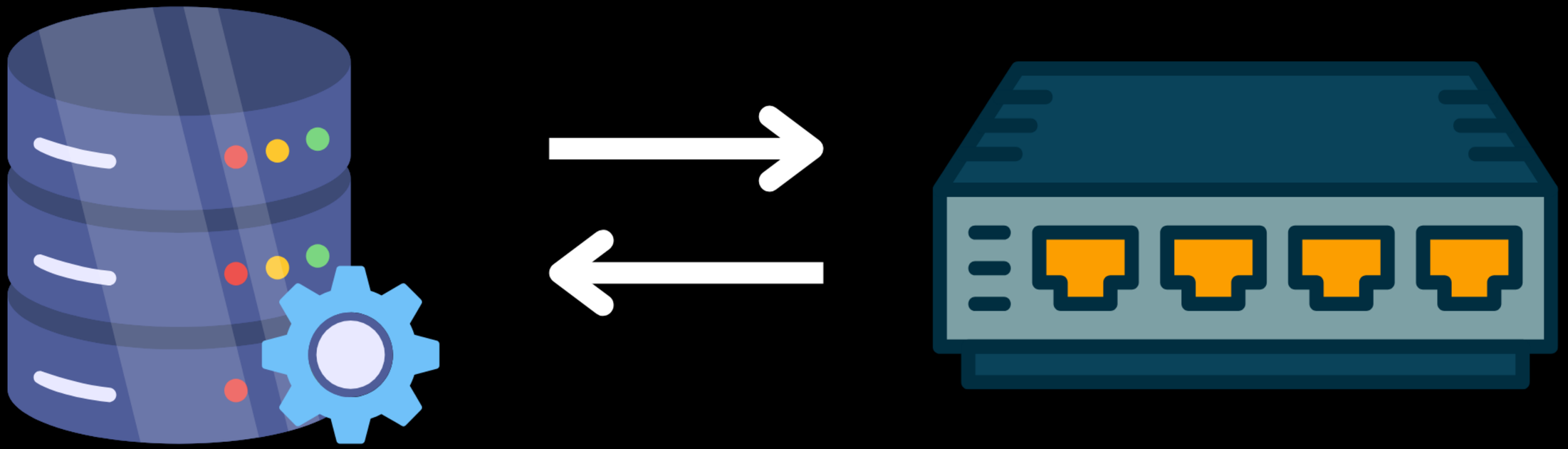
CAT Fuzzing Architecture



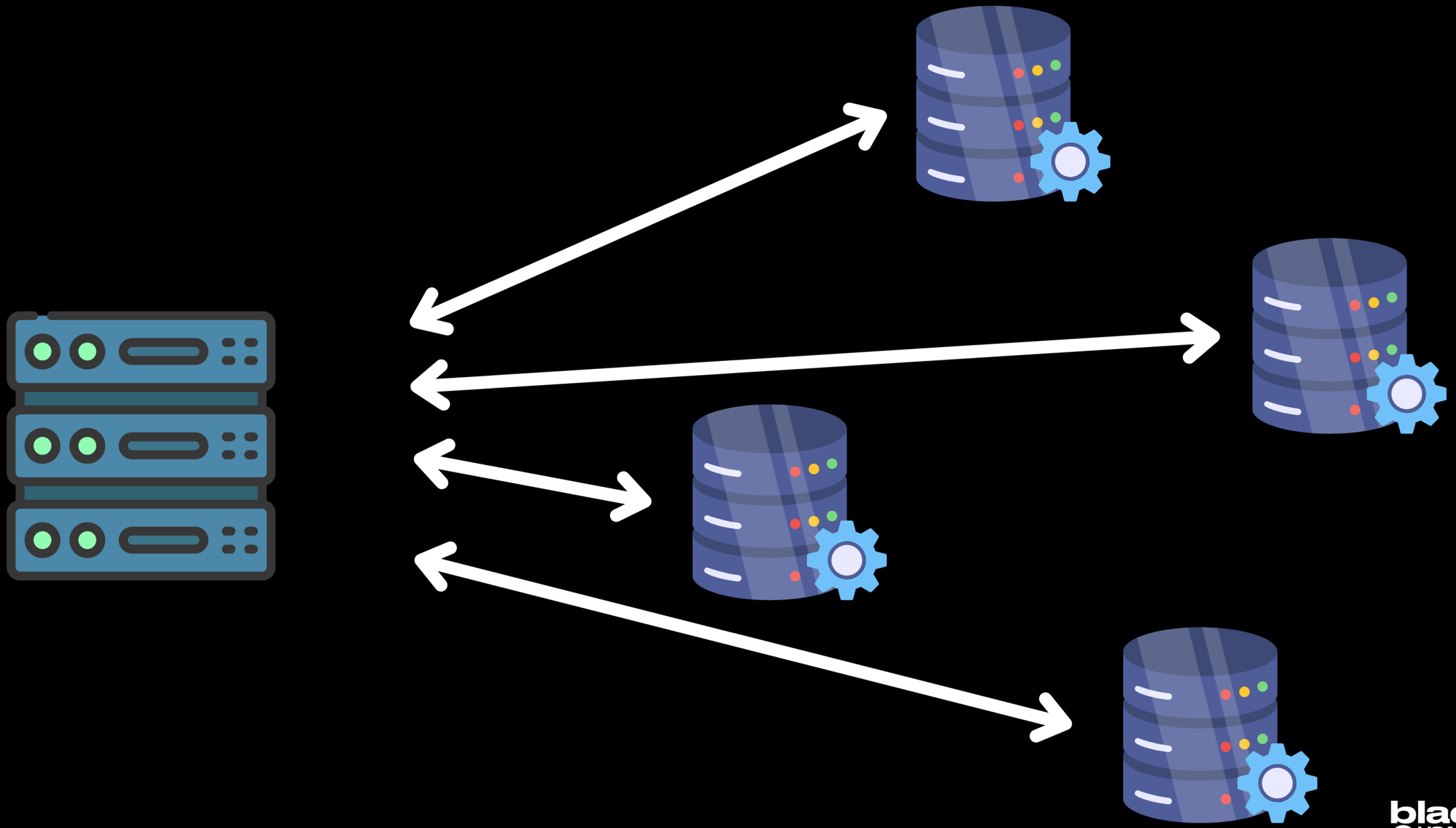
5 - Findings
Vulnerabilities and CVEs

But first...
The RPKI threat model

RPKI requires availability



RPKI is easy to attack globally



5 - Vulnerabilities in RPKI

5 implementations, 21 vulnerabilities, 8 CVEs

Vulnerabilities in RPKI

Type	Amount	Severity	CVEs	Implementations
Remote Code Execution	1	9.8 (critical)	CVE-2024-45237	Fort Validator
Cache Poisoning	1	-	-	OctoRPKI
Denial of Service	19	7.5 (high)	CVE-2025-0638, CVE-2024-45238, CVE-2024-45235, CVE-2024-45236, CVE-2024-45239, CVE-2024-45234, CVE-2024-56375	Routinator, rpki-client, Fort Validator, Prover, OctoRPKI

Remote Code Execution

2 byte buffer

```
unsigned char data[2];

if (ku->length == 0) {
    return pr_val_err("%s bit string has no enabled bits.",
        ext_ku()->name);
}

memset(data, 0, sizeof(data));
memcpy(data, ku->data, ku->length);
```

memcpy with attacker controlled length

More on this in
https://dl.acm.org/doi/abs/10.1145/3658644.3691387

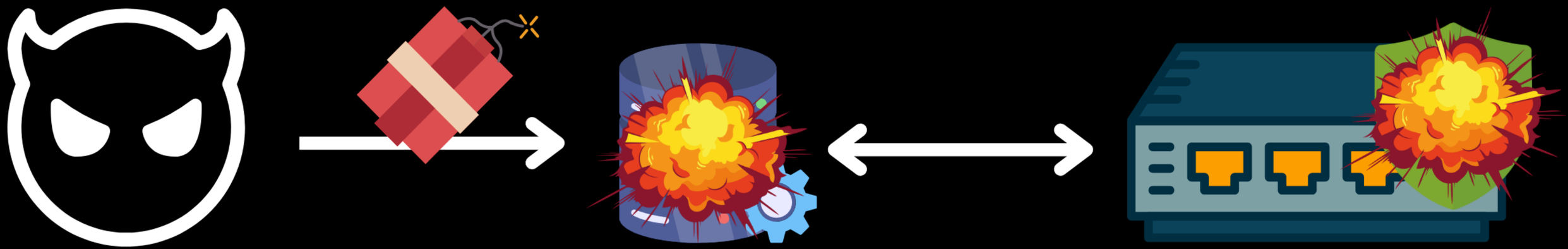
Remote Code Execution

Cache Poisoning

✓ SubjectName: Cloudflare CA
Key: 0x349853

✗ SubjectName: Cloudflare CA
Key: 0xf32856e

Exploiting DoS



ManifestEntries: [0 elements]

Key Takeaways

- Fuzzing in batches is efficient in complex cryptographic infrastructures
- Coverage counters serve as side-channel to map coverage in batches
- Routing security relies on the security of RPKI, which is fragile

Read Paper!

Contact Me!